ALKAID: Accelerating Three-Party Boolean Circuits by Mixing Correlations and Redundancy

Ye Dong, Xudong Chen, Xiangfu Song, Yaxi Yang, Wen-Jie Lu, Tianwei Zhang, *Member, IEEE*, Jianying Zhou, *Fellow, IEEE*, and Jin-Song Dong, *Member, IEEE*

***Abstract*—Secure three-party computation (3PC) with semi-honest security under an honest majority offers notable efficiency in computation and communication; for Boolean circuits, each party sends a single bit for every AND gate, and nothing for XOR. However, round complexity remains a significant challenge, especially in high-latency networks. Some works can support multi-input AND and thereby reduce online round complexity, but they require *exponential* communication for generating the correlations in either preprocessing or online phase. How to extend the AND gate to multi-input while maintaining high correlation generation efficiency is still not solved. To address this problem, we propose a round-efficient 3PC framework ALKAID for Boolean circuits through improved multi-input AND gate. By mixing correlations and redundancy, we propose a concretely efficient correlation generation approach for small input bits $N < 4$ and shift the correlation generation to the preprocessing phase. Building on this, we create a round-efficient AND protocol for general cases with $N > 4$. Exploiting the improved multi-input AND gates, we design fast depth-optimized parallel prefix adder and share conversion primitives in 3PC, achieved with new techniques and optimizations for better concrete efficiency. We further apply these optimized primitives to enhance the efficiency of secure non-linear functions in machine learning. We implement ALKAID and extensively evaluate its performance. Compared to state of the arts like ABY3 (CCS'2018), Trifecta (PoPETs'2023), and METEOR (WWW'2023), ALKAID enjoys 1.5×–2.5× efficiency improvements for boolean primitives and non-linear functions, with better or comparable communication.**

***Index Terms*—Privacy, security, secure three-party computation (3PC), boolean circuits.**

I. INTRODUCTION

SECURE multiparty computation (MPC) enables two or more parties to compute a function on their joint data. It ensures that no party can learn anything about other's inputs, except what can be inferred from outputs [1], [2], [3]. MPC constructions have been applied in various real-world scenarios, e.g., financial analysis [4], [5], medical data computation [6], and machine learning [7], [8], [9], [10]. Specifically, the 3-party with honest majority setting is one of the most promising solutions for practical privacy-preserving applications due to its attractive balance between efficiency and security, and it has been widely adopted in many privacy-preserving applications, including machine/deep learning [9], [11], [12], decision tree [13], [14], [15], graph analysis [16], and private inference of large language models [17], [18], and has been included in mainstream open-sourced frameworks [10], [19].

Existing MPC protocols fall into two primary categories: Yao's garbled circuits (GC) [1], [2], [20] and secret sharing-based protocols [3], [21]. In the GC-based approach, the parties use a garbled Boolean circuit to evaluate the function with constant rounds of communication. On the other hand, secret sharing-based protocols secret-share inputs among parties and compute circuits layer-by-layer, requiring multiple rounds of interaction. GC-based protocols are advantageous in high-latency networks because of their constant round communication, but the communication overhead of exchanging garbled Boolean circuits significantly impacts their throughput. In contrast, secret sharing-based protocols have lightweight communication costs and can be highly parallelized [21].

The need for multiple rounds of communication to compute Boolean gates AND is the main performance bottleneck of secret sharing-based protocols, even for the cheapest 3PC solutions. Those existing works are primarily designed for secure 2-input AND gate, and circuit optimizations and parallelization are typically aligned with this designation, leading to at least $\log_2(\ell)$ rounds of online communication for circuits of depth ℓ. This round complexity is particularly problematic for real-time applications in high-latency networks, where excessive communication rounds can severely restrict the throughput. If a *multi-input* AND gate could be securely computed with comparable (online) round complexity to 2-input ones, the circuit depth will be further reduced, resulting in significant online time savings. Consequently, *it is crucial to propose a round-efficient approach to compute multi-input* AND *gate securely, to improve the performance, especially the online time in high-latency networks, for secret sharing-based protocols.*

A. Existing Solutions

In recent years, there has been increasing research [22], [23], [24], [25] aiming to reduce the *online round complexity* of secret sharing-based protocols. They exploit correlations to

Received 14 June 2025; revised 3 December 2025; accepted 20 December 2025. Date of publication 24 December 2025; date of current version 7 January 2026. This work was supported in part by the National Research Foundation, Singapore; in part by the Cyber Security Agency of Singapore under its National Cybersecurity Research and Development Programme and CyberSG Research and Development Cyber Research Programme Office; and in part by NSFC, China, under Grant 62302118. The associate editor coordinating the review of this article and approving it for publication was Dr. Wenhai Sun. *(Corresponding author: Xiangfu Song.)*

Ye Dong and Jin-Song Dong are with the School of Computing, National University of Singapore, Singapore 117417 (e-mail: dongye@nus.edu.sg; dcsdjs@nus.edu.sg).

Xudong Chen is with the Institute of Information Engineering, Chinese Academy of Sciences, Beijing 100085, China (e-mail: chenxudong@iie.ac.cn).

Xiangfu Song and Tianwei Zhang are with the College of Computing and Data Science, Nanyang Technological University, Singapore 639798 (e-mail: xiangfu.song@ntu.edu.sg; tianwei.zhang@ntu.edu.sg).

Yaxi Yang and Jianying Zhou are with Singapore University of Technology and Design, Singapore 487372 (e-mail: yaxi_yang@sutd.edu.sg; jianying_zhou@sutd.edu.sg).

Wen-Jie Lu is with TikTok Inc., Singapore 048583 (e-mail: luwenjie@tiktok.com).

Digital Object Identifier 10.1109/TIFS.2025.3648188

1556-6021 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Nanyang Technological University Library. Downloaded on January 25,2026 at 03:17:03 UTC from IEEE Xplore. Restrictions apply.

TABLE I

EXISTING SOLUTIONS AND ALKAID FOR ANDING ℓ BITS. • REPRESENTS SECRET SHARING-FULLY SCHEMES AND ○ DENOTES DEALER-BASED APPROACHES.COMMUNICATION IS MEASURED BY THE BITS SENT BY ALL PARTIES.N IS SET AS THE MAXIMUM RECOMMENDED BY EACH WORK.PREP. IS FOR PREPROCESSING AND COMM. DENOTES COMMUNICATION. κ IS THE SECURITY PARAMETER WITH $\kappa = 128$

Framework	# Party	Type	N	ℓ-bit AND			
				Round [Prep.]	Comm. [Prep.]	Comm. [Online]	Round [Online]
[23]	$(2+1)$PC	○	$N \leq 9$	1	$\approx 125\ell$	$\ell/4$	$\log_9(\ell)$
Trifecta [25]	$(2+1)$PC	○	$N \leq 8$	0	0	$\approx 71\ell$	$\log_8(\ell)$
ABY3 [9]	3PC	•	$N = 2$	0	0	3ℓ	$\log_2(\ell)$
METEOR [24]	3PC	•	$N \leq 4$	2	11ℓ	ℓ	$\log_4(\ell)$
ALKAID (Ours)	3PC	•	$N \leq 4$	1	2ℓ	2ℓ	$\log_4(\ell)$

design secure *multi-input* AND gates, which are applied to construct depth-optimized circuits for better online efficiency. At a high-level, given N inputs $\{x_j\}_{j=1}^N$, a multi-input AND follows:

$$-12pt \bigwedge_{j=1}^{N} x_j = \bigwedge_{j=1}^{N} \big((x_j \oplus r_j) \oplus r_j\big)$$
$$= \bigoplus_{\mathcal{T} \subseteq \{1,\ldots,N\}} \big(\bigwedge_{j \notin \mathcal{T}} \underbrace{(x_j \oplus r_j)}_{m_j}\big) \wedge \big(\underbrace{\bigwedge_{k \in \mathcal{T}} r_k}_{r_{\mathcal{T}}}\big), \quad (1)$$

where $\{r_j\}_{j=1}^N$ and $r_{\mathcal{T}} = \bigwedge_{k \in \mathcal{T}} r_k$ for $\forall \mathcal{T} \subseteq \{1, \ldots, N\}$ are kept secret, and $m_j = x_j \oplus r_j$ is revealed. After generating all correlated randomness $r_{\mathcal{T}}$, the parties can compute $\bigwedge_{j=1}^N x_j$ securely in one round of communication. As m_j is public, the key challenge is computing $\{r_{\mathcal{T}}\}_{\mathcal{T} \subseteq \{1,\ldots,N\}}$ securely and efficiently. Based on the generation methods, existing works can be categorized into two approaches:

i) *Dealer-based* protocols [23], [25], [26] rely on a trusted dealer to generate and distribute correlated randomness. This requires the dealer to send $O(2^N)$ bits in one round and have full access to correlated randomness, so it necessarily assumes that the dealer is a trusted party, which might be unavailable in the real world.

ii) *Secret sharing-fully* schemes [22], [24] allow the parties to independently sample randomness r_j within their respective secret-sharing schemes. Correlated values $r_{\mathcal{T}}$ are computed by executing underlying 2-input AND protocols. This method not only incurs a communication of $O(2^N)$ bits but also introduces an additional round complexity of $\log_2(N)$. Unlike dealer-based approaches, secret sharing-fully methods ensure that no single party gets the full correlations in cleartext, making them more suitable when trusted dealer assumption is unavailable.

On the other hand, [22], [23], [24], [26] shift correlation generation to the preprocessing phase to reduce online costs. While these techniques do reduce online round complexity, *they still involve an expensive preprocessing communication complexity of* $O(2^N)$ *(and secret sharing-fully methods require preprocessing round complexity of* $\log_2(N)$*)*. To balance the preprocessing and online communication, they all restrict N to a specific bound, *i.e.*, $N \leq 4$. We summarize N limitations, concrete communication and round complexity for ANDing ℓ bits of existing works in Table I. Given the considerable correlation generation costs and the trust assumptions of current multi-input AND protocols, we ask the following question:

Can we reduce the preprocessing communication and round complexity for generating correlations of multi-input AND *while maintaining (comparable) current high online efficiency in the secret sharing-fully setting?*

To answer this question affirmatively, we propose ALKAID, a secure three-party computation framework against semi-honest adversaries. We mix the correlations of random masks and redundancy of replicated secret sharing to balance concrete preprocessing and online communication costs of N-input AND gates for $N \leq 4$. With its guidance, we design efficient 3PC protocols for depth-optimized circuits, including parallel prefix adder (PPA), share conversions, and bit extraction, which finally improve the efficiency of the non-linear function of secure neural network inference. In summary, ALKAID includes the following contributions:

- **Cost-Balanced N-Input AND with $N \leq 4$.** We propose practical 3-party protocols for N-input AND with $N \leq 4$. Our approach leverages correlations and redundancy of replicated secret sharing to balance preprocessing and online costs. The preprocessing and online phases merely require *one round of communication with* ≤ 2 *bits per party*, making it suitable for high-latency networks. Compared to prior works [9], [24], [25], [26], we achieve an improved trade-off between preprocessing and online costs.
- **Optimized PPA Circuit & Primitives.** We design an efficient protocol for depth-optimized parallel prefix adder (PPA) [22] in 3PC using our N-input AND gates. By exploiting the properties of operators within PPA, we propose specific optimizations to reduce online and preprocessing communication by at least $2\times$ and $1.5\times$, respectively. Since PPA facilitates a bundle of operations in secure computation, our depth-optimized PPA improves the efficiency of widely used 3PC primitives like Arithmetic-Boolean share conversions and bit extraction. We present the dependency of the protocols in Fig. 1.
- **Application & Evaluation.** We implement our protocols in C++ and integrate them into 3PC backend of SecretFlow-SPU [10] to provide our framework ALKAID, and apply it for activation functions of secure neural network inference, handling specific input and output conversions to facilitate its use in Boolean circuits. Exper-

Authorized licensed use limited to: Nanyang Technological University Library. Downloaded on January 25,2026 at 03:17:03 UTC from IEEE Xplore. Restrictions apply.

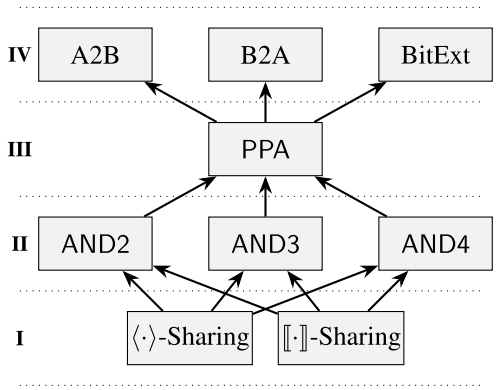


Fig. 1. Dependency of protocols in ALKAID. Layer I specifies the sharing semantics, layer II presents the ($N \leq 4$)-input AND protocols, layer III corresponds to our depth-optimized PPA, and layer IV contains the basic 3PC primitives.

imental results demonstrate that ALKAID accelerates the online phase of 3PC and offers a more balanced trade-off between preprocessing and online phases: ALKAID i) achieves a speedup of 1.5–2.5× compared to [9], [24], and [25] for online efficiency of non-linear functions, and ii) even outperforms one of the cheapest solution ABY3 by at least 7% for end-to-end running time in secure evaluation of neural networks and GPT-2. The source code is available: https://github.com/CPS4AI/OpenAlkaid.

B. Organization

We first introduce the background and preliminary in § II. Then, we present our intuition and techniques for N-input AND gates in § III. Next, the constructions of depth-optimized PPA and 3PC primitives are illustrated in § IV. Afterward, we apply our techniques in the secure non-linear functions of neural network inference in § V. In § VI, we implement our framework and conduct experimental evaluations. Finally, we summarize the related works in § VII and conclude this work in § VIII.

II. Background & Preliminary

A. Notations

We summarize the main notations in Table II. By default, we use the above notations for Boolean sharing, and $[\cdot]^{\mathsf{A}}$, $\langle\cdot\rangle^{\mathsf{A}}$, and $[\![\cdot]\!]^{\mathsf{A}}$ are for the corresponding Arithmetic sharing.

B. Three-Party Computation

We introduce the secret sharing schemes and randomness generation procedures as follows.

1) Linear Secret Sharing (LSS, $[\cdot]$-Sharing): A secret value $x \in \mathbb{Z}_2$ is shared by three random values $r \xleftarrow{\$} \mathbb{Z}_2$, $r' \xleftarrow{\$} \mathbb{Z}_2$, and $r'' = x \oplus r \oplus r'$, where $\mathcal{P}_0$ gets $[x]_0 = r$, $\mathcal{P}_1$ gets $[x]_1 = r'$, and $\mathcal{P}_2$ obtains $[x]_2 = r''$. We refer to it as *Boolean Sharing*. When $x \in \mathbb{Z}_{2^\ell}$ with $\ell > 1$ (*e.g.*, $\ell = 64$), which support *Arithmetic* operations (*e.g.*, $+$, $-$, and $\cdot$) over $\mathbb{Z}_{2^\ell}$, this is *Arithmetic Sharing* and we use notation $[\cdot]^{\mathsf{A}}$.

TABLE II
NOTATION TABLE

Symbols	Descriptions
$\mathcal{P}_i$	party i in 3PC
x	lowercase letter denotes scalar
x_j	the j-th bit of x
$\oplus, \wedge$	bit-wise XOR and AND
$[\cdot]$	3-party linear secret sharing
$\langle\cdot\rangle$	2-out-of-3 replicated secret sharing
$[\![\cdot]\!]$	2-out-of-3 masked replicated secret sharing
AND2/ANDN	AND gate with $2/N$ inputs
$\mathcal{F}_f$	the ideal functionality of function f
PPA	Parallel Prefix Adder

a) XOR: For LSS, we use its secure XOR. Let $a, b, c \in \mathbb{Z}_2$ be public constants, $[x]$ and $[y]$ be two secret-shared inputs over $\mathbb{Z}_2$, $[ax \oplus by \oplus c]$ can be computed as $(a[x]_0 \oplus b[y]_0 \oplus c, a[x]_1 \oplus b[y]_1, a[x]_2 \oplus b[y]_2)$, where $\mathcal{P}_i$ can compute its share locally. When $a = 1, b = 1$, and $c = 0$, we get $[x \oplus y]$.

2) Replicated Secret Sharing (RSS, $\langle\cdot\rangle$-Sharing): RSS is constructed on LSS with redundancy. Given secret $x \in \mathbb{Z}_2$, it is also shared by three random values $[x]_0, [x]_1, [x]_2 \in \mathbb{Z}_2$ with $x = [x]_0 \oplus [x]_1 \oplus [x]_2$. $\mathcal{P}_i$ gets two of the three random values as its RSS share, *a.k.a.*, $\langle x\rangle_i = ([x]_i, [x]_{i+1})$.

a) XOR & AND: Let $a, b, c \in \mathbb{Z}_2$ be public constants, $\langle x\rangle$ and $\langle y\rangle$ be two secret-shared inputs. $\langle ax \oplus by \oplus c\rangle$ can be computed as $(a[x]_0 \oplus b[y]_0 \oplus c, a[x]_1 \oplus b[y]_1, a[x]_2 \oplus b[y]_2)$, where P_i can compute its share locally. When $c_1 = 1, c_2 = 1$, and $c_3 = 0$, we get $\langle x \oplus y\rangle$. On the other hand, computing secure $\langle x \wedge y\rangle$ requires communication among the parties: i) $\mathcal{P}_i$ first computes $[z]_i = [x]_i[y]_i \oplus [x]_{i+1}[y]_i \oplus [x]_i[y]_{i+1}$ locally, ii) Parties then perform *re-sharing* by letting $\mathcal{P}_i$ send $[z']_i = [\alpha]_i \oplus [z]_i$ to $\mathcal{P}_{i-1}$, where $[\alpha]_0 \oplus [\alpha]_1 \oplus [\alpha]_2 = 0$ and are generated by functionality $\mathcal{F}_{\mathsf{ZeroShr}}$ (c.f., § II-C). In the end, $([z']_0, [z']_1), ([z']_1, [z']_2), ([z']_2, [z']_0)$ form $\langle x \wedge y\rangle$.

3) Masked Replicated Secret Sharing (MRSS, $[\![\cdot]\!]$-Sharing): In MRSS [24], $x \in \mathbb{Z}_2$ is shared as $[\![x]\!] = (m_x, \langle r_x\rangle)$ where: i) r is random sampled from $\mathbb{Z}_2$ and RSS-shared among parties, and ii) $m_x = x \oplus r_x$ is revealed to all parties. As no party knows r_x in clear (except the secret owner), revealing m_x will not introduce any leakage of x. MRSS utilizes the same secret-randomness $\langle r\rangle$ to protect inputs and generate correlations.

a) XOR & AND: Let a, b, and c be public constants, $[\![x]\!]$ and $[\![y]\!]$ be two secret inputs, where $[\![x]\!] = (m_x, \langle r_x\rangle)$ and $[\![y]\!] = (m_y, \langle r_y\rangle)$. It is easy to see that the parties can compute $(am_x \oplus bm_y \oplus c, a\langle r_x\rangle \oplus b\langle r_y\rangle)$ locally. Also, we get $[\![x \oplus y]\!]$ with $a = 1, b = 1$, and $c = 0$. When computing MRSS-based AND2$[\![x \wedge y]\!]$, the parties work in a *preprocessing/online* paradigm: i) Preprocessing: parties compute $\langle r_{xy}\rangle = \langle r_x\rangle \wedge \langle r_y\rangle$ using RSS-based AND2. ii) Online: parties compute $\langle m_z\rangle = m_x m_y \oplus m_x\langle r_y\rangle \oplus m_y\langle r_x\rangle \oplus \langle r_{xy}\rangle \oplus \langle r_z\rangle$, reveal m_z to all parties, and set the result as $(m_z, \langle r_z\rangle)$, where $\langle r_z\rangle$ is random sampled and $\langle\cdot\rangle$-shared. Dong et al. [24] proposed MRSS-based ANDN gates with improved online efficiency with exponential preprocessing communication.

C. Security Model & Ideal Functionalities

Following prior works [9], [24], ALKAID is secure against a semi-honest (*a.k.a.*, honest-but-curious) adversary that

Authorized licensed use limited to: Nanyang Technological University Library. Downloaded on January 25,2026 at 03:17:03 UTC from IEEE Xplore. Restrictions apply.

corrupts no more than one of the three computing parties. Semi-honest means such an adversary will follow the protocol specifications, but may try to learn other's private information.

Definition 1 (Semi-Honest Security): Let Π be a three-party protocol running in real-world and $\mathcal{F} : (\{0,1\}^n)^3 \rightarrow (\{0,1\}^m)^3$ be the ideal randomized functionality. We say Π securely computes $\mathcal{F}$ against a single semi-honest adversary if for every corrupted party $\mathcal{P}_i$ ($i \in \{0,1,2\}$) and every input $x \in (\{0,1\}^n)^3$, there exists an efficient simulator $\mathcal{S}$:

$$\{\mathsf{view}_{i,\Pi}(x), \mathsf{output}_{\Pi}(x)\} \overset{c}{\approx} \{\mathcal{S}(i, x_i, \mathcal{F}_i(x)), \mathcal{F}(x)\},$$

where $\mathsf{view}_{i,\Pi}(x)$ is the view of $\mathcal{P}_i$ in the execution of Π on $\mathbf{x}$, $\mathsf{output}_{\Pi}(x)$ is the output of all parties, and $\mathcal{F}_i(x)$ denotes the i-th output of $\mathcal{F}(x)$.

1) Ideal Functionalities: We use several well-established functionalities. The implementations of $\mathcal{F}_{\mathsf{AND2}}^{\langle\cdot\rangle}$ and $\mathcal{F}_{\mathsf{AND2}}^{[\![\cdot]\!]}$ are as § II-B2 and § II-B3, respectively. Below, we introduce the randomness generation functionalities.

2) Randomness Generation: We rely on that parties can generate fresh random elements on demand, without any interaction beyond a short initial phase. Concretely, i) all parties have a common key, ii) $\mathcal{P}_i$ obtains random keys $(\mathsf{key}_i, \mathsf{key}_{i+1})$, such that each pair $(\mathcal{P}_i, \mathcal{P}_{i+1})$ have the common key_{i+1}. We use the following pseudorandom function (PRF)-based randomness generation procedures from [9] and [27]:

- Π_{Rand}: Given a fresh id, $\mathcal{P}_i$ lets $[r]_i = \mathsf{PRF}_{\mathsf{key}_i}(\mathsf{id})$ and $[r]_{i+1} = \mathsf{PRF}_{\mathsf{key}_{i+1}}(\mathsf{id})$ so that $([r]_0, [r]_1, [r]_2)$ form valid RSS shares of random r;
- Π_{RandComm}: Each $\mathcal{P}_i$ lets $r = \mathsf{PRF}_{\mathsf{key}}(\mathsf{id})$ for the fresh id, so that all parties have a common random r;
- $\Pi_{\mathsf{RandPair}_{i,j}}$: Each pair of $\mathcal{P}_i$ and $\mathcal{P}_j$ locally computes $r = \mathsf{PRF}_{\mathsf{key}}(\mathsf{id})$, where key is their common key. Therefore, $\mathcal{P}_i$ and $\mathcal{P}_j$ have common random r.
- Π_{ZeroShr}: Each $\mathcal{P}_i$ computes $[\alpha]_i = \mathsf{PRF}_{\mathsf{key}_i}(\mathsf{id}) \oplus \mathsf{PRF}_{\mathsf{key}_{i+1}}(\mathsf{id})$. In this way, $[\alpha]_0 \oplus [\alpha]_1 \oplus [\alpha]_2 = 0$.

PRF is defined as $\mathsf{PRF} : \{0,1\}^\kappa \times \{0,1\}^\kappa \rightarrow \{0,1\}^\ell$, where κ is the computational security parameter, *a.k.a.*, the length of the keys. When requiring a random bit-string, we denote it as $r \overset{\$}{\leftarrow} \mathbb{Z}_2^\ell$. Similarly, when needing an ℓ-bits random value from $\mathbb{Z}_{2^\ell}$, we interpret it as $r \overset{\$}{\leftarrow} \mathbb{Z}_{2^\ell}$ (and $\oplus$ can be replaced by $+$ or $-$ modulo 2^ℓ).

Note: To achieve information-theoretic security, the parties can locally sample uniform random values and communicate with others to generate correlated randomness. More details can be referred to [27] and [28].

D. Preprocessing/Online Paradigm

The preprocessing/online paradigm is widely used in designing practical MPC protocols. Recent works [22], [23], [24], [43] make use of input-independent but function-dependent preprocessing phases to further improve online efficiency. This approach necessitates that the parties know the function during preprocessing, which is often met with basic MPC primitives (*e.g.*, share conversions) and some real-world applications like secure neural network inference. We employ an input-independent but function-dependent preprocessing phase.

III. Intuition & Technique for ANDN

We first revisit the RSS and MRSS-based approaches for ANDN, then present the intuitions and techniques, and finally show our concrete ANDN design.

A. Revisiting RSS & MRSS-Based Approaches

We revisit the secure ANDN of both approaches and analyze their respective preprocessing and online complexity.

1) RSS-Based Approach: RSS-based works [9], [11], [27], [28] use the *redundancy* to compute AND2 securely. As discussed in § II-B2, each AND2 needs one re-sharing to maintain the consistency of sharing semantics for subsequent computation. Re-sharing requires one party to send 1 bit in 1 round. When computing the ANDN gate, these works exploit AND2 and *binary tree*-based optimization to represent ANDN as a binary tree of depth $\log_2(N)$, so parties can compute AND2 gates layer-by-layer: in the i-th layer, the parties compute $N/2^i$ AND2 gates in parallel (*a.k.a.*, 1 round) with communicating $3N/2^i$ bits totally. This method requires all parties to send around $3N$ bits in $\log_2(N)$ rounds.

2) MRSS-Based Approach: MRSS-based approaches work in the preprocessing/online paradigm and focus on optimizing online complexity [24]. As discussed in § II-B3, parties compute correlations in the preprocessing phase and consume them in the online phase. This approach can be generally extended to ANDN: Given N secret values $\{[\![x_j]\!] = (m_{x,j}, \langle r_{x,j}\rangle)\}_{j=1}^N$, parties first generate all correlated randomness $\langle r_{\mathcal{T}}\rangle = \wedge_{k\in\mathcal{T}}\langle r_{x,k}\rangle$ for $\mathcal{T} \subseteq \{1,\ldots,N\}$ using RSS-based AND2, then compute $\langle m_z\rangle = \bigoplus_{\mathcal{T}\subseteq\{1,\ldots,N\}}(\wedge_{j\notin\mathcal{T}} m_j) \wedge \langle r_{\mathcal{T}}\rangle \oplus \langle r_z\rangle$, and finally reveal m_z to all parties. The result is $[\![z]\!] = (m_z, \langle r_z\rangle)$ with $z = \wedge_{j=1}^N x_j$. Asymptotically, this approach requires each party to send 1 bit in 1 round during the online phase, but with communicating $(2^N - N - 1)$ bits in $\log_2(N)$ rounds for the preprocessing. It sets $N \leq 4$ to balance preprocessing and online costs and constructs QuadTree for $N > 4$: all parties send N bits in $\log_4(N)$ rounds in online phase, with communicating $11N$ bits in 2 preprocessing rounds.

The analysis illustrates that RSS-based ANDN does not need any preprocessing communication, but it has higher online communication and round complexity than the MRSS-based approach. However, the preprocessing communication complexity of the MRSS-based method is exponential to N so we have to set $N \leq 4$.

B. Our Intuition & Technique

In MPC, we usually guarantee the sharing consistency of inputs and outputs for all gates to support subsequent computation. For example, when computing ANDN, the RSS-based method requires re-sharing to convert intermediate shares back into the RSS format, and the MRSS-based approach needs to reveal the masked values to get the MRSS-shared results. However, when we go beyond the *layer-by-layer* paradigm and look at more layers of gates as a whole, we observe that *it is sufficient to guarantee the sharing consistency of*

Authorized licensed use limited to: Nanyang Technological University Library. Downloaded on January 25,2026 at 03:17:03 UTC from IEEE Xplore. Restrictions apply.

Protocol Π_{AND4}

Inputs: For $i \in \{0,1,2\}$, $\mathcal{P}_i$ inputs $[\![x]\!]_i = (m_x, \langle r_x \rangle_i)$, $[\![y]\!]_i = (m_y, \langle r_y \rangle_i)$, $[\![u]\!]_i = (m_u, \langle r_u \rangle_i)$, and $[\![v]\!]_i = (m_v, \langle r_v \rangle_i)$.
Outputs: $\mathcal{P}_i$ gets $[\![z]\!]_i = (m_z, \langle r_z \rangle_i)$ with $z = x \wedge y \wedge u \wedge v$.

Preprocessing Phase:

1) Parties compute correlated randomness $\langle r_{xy} \rangle = \langle r_x \rangle \wedge \langle r_y \rangle$ and $\langle r_{uv} \rangle = \langle r_u \rangle \wedge \langle r_v \rangle$ by using $\mathcal{F}_{\mathsf{AND2}}^{\langle \cdot \rangle}$.
2) Parties invoke $\mathcal{F}_{\mathsf{ZeroShr}}$ to let $\mathcal{P}_i$ obtain $[\alpha]_i$ with $[\alpha]_0 \oplus [\alpha]_1 \oplus [\alpha]_2 = 0$.
3) Parties invoke $\mathcal{F}_{\mathsf{Rand}}$, such that $\mathcal{P}_i$ gets random share $\langle r_z \rangle_i = ([r_z]_i, [r_z]_{i+1})$.

Online Phase:

1) Parties compute $\langle xy \rangle = m_x m_y \oplus m_x \langle r_y \rangle \oplus m_y \langle r_x \rangle \oplus \langle r_{xy} \rangle$ and $\langle uv \rangle = m_u m_v \oplus m_u \langle r_v \rangle \oplus m_v \langle r_u \rangle \oplus \langle r_{uv} \rangle$ locally, such that $\mathcal{P}_i$ gets $\langle xy \rangle_i = ([xy]_i, [xy]_{i+1})$ and $\langle uv \rangle_i = ([uv]_i, [uv]_{i+1})$.
2) $\mathcal{P}_i$ locally computes $[z]_i = ([xy]_i \wedge [uv]_i) \oplus ([xy]_{i+1} \wedge [uv]_i) \oplus ([xy]_i \wedge [uv]_{i+1})$.
3) $\mathcal{P}_i$ computes $[m_z]_i = [z]_i \oplus [\alpha]_i \oplus [r_z]_i$ and sends $[m_z]_i$ to $\mathcal{P}_{i-1}$ and $\mathcal{P}_{i+1}$, such that each party can reconstruct $m_z = \oplus_{i=0}^{2} [m_z]_i$.
4) $\mathcal{P}_i$ sets $[\![z]\!]_i = (m_z, \langle r_z \rangle_i)$ as outputs.

Fig. 2. Protocol for secure ANDing 4 bits.

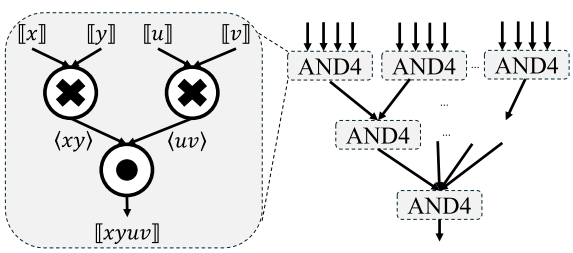


Fig. 3. The illustration of our N-input AND protocol. $\otimes$ denotes the modified MRSS-based AND_2, and $\odot$ denotes RSS-based AND_2. We assume $N = 4^n$ and only need to ensure the sharing consistency of the inputs and the final output.

some necessary gates, but not all, to compute ANDN *in 3PC*. As illustrated in Fig. 3, we modify the MRSS-based AND2 to generate RSS-shared intermediate results without communication, which support one more layer of RSS-based AND2 followed by one resharing. Therefore, we only need to guarantee the sharing consistency of inputs and final outputs in ANDing 4 bits, saving overhead for processing the sharing semantics of intermediate results.

With this insight, we make use of the correlations of MRSS and redundancy of RSS to compute ANDN for $N \leq 4$. Let $N = 4$, and the secret inputs $[\![x]\!] = (m_x, \langle r_x \rangle), [\![y]\!] = (m_y, \langle r_y \rangle), [\![u]\!] = (m_u, \langle r_u \rangle)$, and $[\![v]\!] = (m_v, \langle r_v \rangle)$ are MRSS-shared inputs. We build our protocol Π_{AND4} using the variants of AND2 of MRSS and RSS as follows:

- **Preprocessing:** In the preprocessing phase, parties first compute correlated randomness $\langle r_{xy} \rangle = \langle r_x \rangle \wedge \langle r_y \rangle$ and $\langle r_{uv} \rangle = \langle r_u \rangle \wedge \langle r_v \rangle$ using RSS-based AND2. Additionally, parties generate zero-sharing $([\alpha]_0, [\alpha]_1, [\alpha]_2)$ and RSS-shared random $\langle r \rangle$, where $\mathcal{P}_i$ has $[\alpha]_i$ and $\langle r \rangle_i$.
- **Online:** We present our mixing correlation and redundancy as follows. i) **Correlations:** instead of directly computing MRSS-shared results, we modify the AND2 of MRSS to output RSS-shared results utilizing the correlated randomness, *i.e.*, $\langle xy \rangle = m_x m_y \oplus m_x \langle r_y \rangle \oplus m_y \langle r_x \rangle \oplus \langle r_{xy} \rangle$, so that the parties can compute $\langle xy \rangle$ and $\langle uv \rangle$ locally (free of communication). ii) **Redundancy:** we observe that $\langle xy \rangle$ and $\langle uv \rangle$ are still of RSS-shared format, so we can exploit the *redundancy* of RSS to compute one AND2. Concretely, parties compute the 3-out-of-3 shares of $z = (xy) \wedge (uv)$ locally, where $\mathcal{P}_i$ holds $[z]_i$ and $z = [z]_0 \oplus [z]_1 \oplus [z]_2$. Thirdly, $\mathcal{P}_i$ computes and sends $[m_z]_i = [z]_i \oplus [\alpha]_i \oplus [r_z]_i$ to $\mathcal{P}_{i-1}$ and $\mathcal{P}_{i+1}$, and all parties can reconstruct $m_z = \bigoplus_{i=0}^{2} [m_z]_i$ to set the result as $[\![z]\!] = (m_z, \langle r_z \rangle)$. In this way, we leverage mixing correlations and redundancy to compute AND4 with one round of communication.

Given the correctness of the MRSS and RSS-based AND2, it is easy to verify $z = x \wedge y \wedge u \wedge v$. As $m_z = \bigoplus_{i=0}^{2} [m_z]_i = \bigoplus_{i=0}^{2} ([z]_i \oplus [\alpha]_i \oplus [r_z]_i) = (\bigoplus_{i=0}^{2} [z]_i) \oplus (\bigoplus_{i=0}^{2} [\alpha]_i) \oplus (\bigoplus_{i=0}^{2} [r_z]_i) = z \oplus 0 \oplus r_z = z \oplus r_z$, we have $[\![z]\!] = (m_z, \langle r_z \rangle)$ as an valid MRSS sharing of $(x \wedge y \wedge u \wedge v)$. Our protocol Π_{AND4} is formulated as Figure 2.

1) Security Analysis: In protocol Π_{AND4}, we aim to compute the ideal functionality $\mathcal{F}_{\mathsf{AND4}}$: It takes four MRSS sharings $[\![x]\!]$, $[\![y]\!]$, $[\![u]\!]$, $[\![v]\!]$ with identifiers id_x, id_y, id_u, id_v, respectively, and outputs $[\![z]\!] = [\![x \wedge y \wedge u \wedge v]\!]$ corresponding to a new identifier id_z. We capture the security in Theorem 1 and give the proof in Appendix E.

Theorem 1: [Security of Π_{AND4}] In the 3-party honest-majority setting, protocol Π_{AND4} securely realizes $\mathcal{F}_{\mathsf{AND4}}$ in the $(\mathcal{F}_{\mathsf{AND2}}^{\langle \cdot \rangle}, \mathcal{F}_{\mathsf{Rand}}, \mathcal{F}_{\mathsf{ZeroShr}})$-hybrid model in the presence of a static semi-honest adversary $\mathcal{A}$ who corrupts at most one single party among three.

2) Communication & Round Complexity: We summarize the communication and round complexity of protocol Π_{AND4} in Lemma 1 and give the proof as follows.

Lemma 1: Protocol Π_{AND4} requires each party to send 2 bits in 1 round during the preprocessing phase, and sends 2 bits in 1 round during the online phase.

Proof: [Proof of Lemma 1.] In the preprocessing phase, the parties run RSS-based AND2 twice in parallel, which requires each party to send a total of 2 bits in 1 round. In the online phase, the parties reveal m_z by every party sending 1 bit to each of the other two parties, a total of 2 bits, in 1 round.■

Compared to the RSS-based solution [9], [27], although we incur some preprocessing costs, we reduce online communication (resp. round) by 1.5× (resp. 2×) for ANDing 4 bits. Compared to the MRSS-based approach [24], we reduce preprocessing communication (resp. round) by 5.5× (resp. 2×) and achieve the same online round complexity and comparable online communication. Our construction achieves a better trade-off of preprocessing and online complexity.

3) Optimizations for $N = 2$ and 3: When ANDing 2 or 3 MRSS-shared inputs, we make the following optimizations:

- **Protocol** Π_{AND2}: To achieve two-input AND functionality $\mathcal{F}_{\mathsf{AND2}}$ with reduced preprocessing costs, we convert the MRSS-shared values into $\langle x \rangle$ and $\langle y \rangle$ locally, and then

Authorized licensed use limited to: Nanyang Technological University Library. Downloaded on January 25,2026 at 03:17:03 UTC from IEEE Xplore. Restrictions apply.

utilize the redundancy of RSS to compute 3-out-of-3 shared $[x \wedge y]$. Finally, we convert the results into MRSS-shared $[\![x \wedge y]\!]$. Compared to [24], this method does not need any preprocessing communication but requires sending 2 bits online. Besides, this results in aligned outputs resharing procedures with Π_{AND4} and Π_{AND3}, and will be helpful when we compute XOR between the results of Π_{AND2} and $\Pi_{\mathsf{AND4}}/\Pi_{\mathsf{AND3}}$. Looking ahead, we will use both Π_{AND2} and MRSS-based AND2 protocol of [24] in designing practical and fast depth-optimized PPA (c.f. § IV-A).

- **Protocol** Π_{AND3}: Given three inputs $[\![x]\!], [\![y]\!], [\![u]\!]$, we adopt a similar approach as Π_{AND4} to achieve functionality $\mathcal{F}_{\mathsf{AND3}}$: we first AND$[\![x]\!]$ and $[\![y]\!]$ using MRSS-based AND2 but keep the result in RSS-shared fashion as $\langle x \wedge y \rangle$ (like step 1) of online phase in protocol Π_{AND4}). At the same time, parties convert $[\![u]\!]$ into RSS sharing $\langle u \rangle$ locally. Finally, all parties compute AND2 gate on $\langle x \wedge y \rangle$ and $\langle u \rangle$ under the realm of RSS and convert the 3-out-of-3 shared $[(x \wedge y) \wedge u]$ into MRSS-shared outputs. In this way, each party only needs to send 1 bit in 1 round during the preprocessing phase and 2 bits in 1 round during the online phase.

Similar to Π_{AND4}, it is easy to see that Π_{AND2} and Π_{AND3} securely realize $\mathcal{F}_{\mathsf{AND2}}$ and $\mathcal{F}_{\mathsf{AND3}}$ in the ($\mathcal{F}_{\mathsf{AND2}}{}^{\langle\cdot\rangle}$, $\mathcal{F}_{\mathsf{Rand}}$, $\mathcal{F}_{\mathsf{ZeroShr}}$)-hybrid model, respectively. The round and communication complexity is analyzed as follows: i) In both protocols, each party sends 2 bits in 1 round for online phase. ii) For the preprocessing phase, Π_{AND2} does not require communication and Π_{AND3} only requires each party to send 1 bit in 1 round.

4) ANDN With $N > 4$: Let n be a parameter and the number of inputs $N = 4^n$. We use protocol Π_{AND4} and QuadTree [29] to build our highly depth-optimized ANDN circuits: Starting from the leaves (*a.k.a.*, inputs), we invoke Π_{AND4} for $N/4^k$ times in parallel (1 round) for the k-th layer AND4 gates, iteratively n sequential layers to reach the root, which gives the final result. When N is not a power of 4, we integrate our protocol Π_{AND2} and Π_{AND3} to process 2- and 3-input AND gates when necessary. We present the detailed protocol with $N > 4$ in Appendix A. It is easy to see the security in the hybrid model. The round and communication complexity is summarized as follows.

Remark 1: Let M_4, M_3, and M_2 denote the numbers of AND4, AND3, and AND2 gates in ANDing N inputs. Each party sends $2M_4 + M_3$ bits in 1 round for the preprocessing phase, and sends $2(M_4 + M_3 + M_2)$ bits in $\lceil \log_4 N \rceil$ rounds during the online phase. Especially, when $N = 4^n$, we only need $\frac{N-1}{3}$ AND4 gates. All parties send $2(N-1)$ bits in 1 round for the preprocessing phase, and send $2(N-1)$ bits in $n = \log_4(N)$ rounds for the online phase.

IV. Improved PPA Circuit & Primitives

We first revisit depth-optimized Parallel Prefix Adder (PPA) circuit (§ IV-A). Then, we apply our improved PPA to design fast 3PC primitives, including Arithmetic-Boolean conversions and most significant bit extraction (§ IV-B and § IV-C).

A. Depth-Optimized PPA & Optimizations

Given two ℓ-bits x and y, the bit-wise representations are $x = x_{\ell-1} \ldots x_1 x_0$ and $y = y_{\ell-1} \ldots y_1 y_0$, where x_j (resp., y_j) denotes the j-th bit of x (resp., y). PPA makes use of the generated (g) and propagated (p) signals to compute the sum of inputs in Boolean circuits [30], [31]. Concretely, given (x_j, y_j), (g_j, p_j) are defined as:

$$g_j = x_j \wedge y_j, \;\; p_j = x_j \oplus y_j. \tag{2}$$

Patra et al. [22] defined operators $\triangledown_N$ and $\blacktriangledown_N$ to compute intermediate signals based on g and p. We revisit $\triangledown_N$ and $\blacktriangledown_N$ for $N = 2, 3, 4$. When $j = 0$, signals can be processed as follows, with indices from 0 to $N-1$:

$$\begin{aligned}
\triangledown_2(g,p) &= g_{j+1} \oplus (p_{j+1} \wedge g_j),\\
\blacktriangledown_2(g,p) &= (g_{j+1} \oplus (p_{j+1} \wedge g_j), p_{j+1} \wedge p_j),\\
\triangledown_3(g,p) &= g_{j+2} \oplus p_{j+2} \wedge (g_{j+1} \oplus (p_{j+1} \wedge g_j))\\
&= g_{j+2} \oplus (p_{j+2} \wedge g_{j+1}) \oplus (p_{j+2} \wedge p_{j+1} \wedge g_j),\\
\blacktriangledown_3(g,p) &= (g_{j+2} \oplus p_{j+2} \wedge (g_{j+1} \oplus (p_{j+1} \wedge g_j)),\\
&\quad p_{j+2} \wedge p_{j+1} \wedge p_j)\\
&\quad = (g_{j+2} \oplus (p_{j+2} \wedge g_{j+1}) \oplus (p_{j+2} \wedge p_{j+1} \wedge g_j),\\
&\quad p_{j+2} \wedge p_{j+1} \wedge p_j),\\
\triangledown_4(g,p) &= g_{j+3} \oplus p_{j+3}\\
&\quad \wedge (g_{j+2} \oplus (p_{j+2} \wedge (g_{j+1} \oplus p_{j+1} \wedge g_j)))\\
&\quad = g_{j+3} \oplus (p_{j+3} \wedge g_{j+2}) \oplus (p_{j+3} \wedge p_{j+2} \wedge g_{j+1})\\
&\quad \oplus (p_{j+3} \wedge p_{j+2} \wedge p_{j+1} \wedge g_j),\\
\blacktriangledown_4(g,p) &= (g_{j+3} \oplus p_{j+3}\\
&\quad \wedge (g_{j+2} \oplus (p_{j+2} \wedge (g_{j+1} \oplus p_{j+1} \wedge g_j))),\\
&\quad p_{j+3} \wedge p_{j+2} \wedge p_{j+1} \wedge p_j)\\
&\quad = (g_{j+3} \oplus (p_{j+3} \wedge g_{j+2}) \oplus (p_{j+3} \wedge p_{j+2} \wedge g_{j+1})\\
&\quad \oplus (p_{j+3} \wedge p_{j+2} \wedge p_{j+1} \wedge g_j),\\
&\quad p_{j+3} \wedge p_{j+2} \wedge p_{j+1} \wedge p_j)
\end{aligned} \tag{3}$$

An example of how to use equation (3) in depth-optimized PPA to compute *carry signals* for 16-bit inputs is illustrated in Figure 6. Following this circuit logical, given ℓ-bit x and y, the high-level evaluation of PPA is as follows:

1) Compute $g_j = x_j \wedge y_j$ and $p_j = x_j \oplus y_j$ for $j \in \{0, 1, \ldots, \ell-1\}$.
2) Recursively compute the intermediate signals layer-by-layer from (p_j, g_j) generated in step 1) using the operators of Equation (3). The generated *carry signals* are $\{c_j\}_{j=0}^{\ell-1}$.
3) For $j \in \{0, 1, \ldots, \ell-1\}$, compute and output the sum bits $s_0 = p_0$ and $s_j = c_{j-1} \oplus p_j$ when $0 < j \leq \ell - 1$.

With our support for 3PC bitwise XOR, AND2, AND3, and AND4, we can securely implement the operators in Equation (3), ultimately achieving 3PC PPA.

1) Optimizations: Leveraging our proposed protocols and operators in Equation (3), we introduce **XOR-then-Resharing**, **AND-Reusing**, and **Mixed-AND2** to reduce the online and preprocessing communication costs in 3PC:

- **XOR-then-Resharing.** As 3-out-of-3 sharing supports XOR, we make use of the XOR-then-Resharing technique

Authorized licensed use limited to: Nanyang Technological University Library. Downloaded on January 25,2026 at 03:17:03 UTC from IEEE Xplore. Restrictions apply.

Protocol Π_{AND2}

Inputs: For $i \in \{0,1,2\}$, $\mathcal{P}_i$ inputs $[\![x]\!]_i = (m_x, \langle r_x \rangle_i)$ and $[\![y]\!]_i = (m_y, \langle r_y \rangle_i)$.
Outputs: $\mathcal{P}_i$ gets $[\![z]\!]_i = (m_z, \langle r_z \rangle_i)$ with $z = x \wedge y$.

Preprocessing Phase:
1) Parties invoke $\mathcal{F}_{\mathsf{ZeroShr}}$ to let $\mathcal{P}_i$ obtain $[\alpha]_i$ with $[\alpha]_0 \oplus [\alpha]_1 \oplus [\alpha]_2 = 0$.
2) Parties invoke $\mathcal{F}_{\mathsf{Rand}}$, such that $\mathcal{P}_i$ gets random share $\langle r_z \rangle_i = ([r_z]_i, [r_z]_{i+1})$.

Online Phase:
1) Parties compute $\langle x \rangle = m_x \oplus \langle r_x \rangle$ and $\langle y \rangle = m_y \oplus \langle r_y \rangle$.
2) $\mathcal{P}_i$ locally computes $[z]_i = ([x]_i \wedge [y]_i) \oplus ([x]_{i+1} \wedge [y]_i) \oplus ([x]_i \wedge [y]_{i+1})$.
3) $\mathcal{P}_i$ computes $[m_z]_i = [z]_i \oplus [\alpha]_i \oplus [r_z]_i$ and sends $[m_z]_i$ to $\mathcal{P}_{i-1}$ and $\mathcal{P}_{i+1}$, such that each party can reconstruct $m_z = \oplus_{i=0}^{2} [m_z]_i$.
4) $\mathcal{P}_i$ sets $[\![z]\!]_i = (m_z, \langle r_z \rangle_i)$ as outputs.

Fig. 4. Protocol for secure ANDing 2 bits.

to reduce the invocations of resharing for $\triangledown_3$, $\blacktriangledown_3$, $\triangledown_4$, and $\blacktriangledown_4$. Taking $\triangledown_3$ as an example, instead of resharing $[p_1 g_2]$ and $[p_1 p_2 g_3]$ into MRSS separately and then computing XOR, parties first compute their XOR, then reshare $[p_1 g_2 \oplus p_1 p_2 g_3]$ as $[\![p_1 g_2 \oplus p_1 p_2 g_3]\!]$, and finally compute $[\![g_1]\!] \oplus [\![p_1 g_2 \oplus p_1 p_2 g_3]\!]$. So, we only reshare *one bit* for $\triangledown$-style operators and the first part of $\blacktriangledown$-style operators, making it *independent of the number of inputs*.

- **AND-Reusing.** Recall that protocols Π_{AND3} and Π_{AND4} generate RSS-shared intermediate results. We can re-use these intermediate results to reduce the preprocessing communication costs of $\blacktriangledown_3$, $\triangledown_4$, and $\blacktriangledown_4$. Taking $\blacktriangledown_3$ as an example, we observe that both $p_1 p_2 g_3$ and $p_1 p_2 p_3$ have term $p_1 p_2$. So we can compute RSS-shared $\langle p_1 p_2 \rangle$ once and use it for both AND3. This saves one invocation of RSS-based AND2 in the preprocessing phase. Also, we can apply similar optimizations to $\triangledown_4$ (resp. $\blacktriangledown_4$) to save one (resp. two) preprocessing RSS-based AND2. We reduce the preprocessing communication costs by at least $1.5\times$ than processing each AND3 and AND4 directly and separately. Since the reused intermediate results are of $\langle\cdot\rangle$-shared fashion and not revealed to any party, this technique will not introduce additional security concerns.
- **Mixed-AND2.** In PPA, we need AND2 to compute $\{[\![g_j]\!]\}_{j=0}^{\ell-1}$ and operators $(\triangledown_N, \blacktriangledown_N)$. When computing $\{[\![g_j]\!]\}_{j=0}^{\ell-1}$, we use the MRSS-based approach [24] so that each party only sends ℓ bits in the online phase. This shifts ℓ bits into the preprocessing phase but does not increase communication rounds. For the AND2 gates involved in $(\triangledown_N, \blacktriangledown_N)$, we apply protocol Π_{AND2} described in § III-B3 because we conduct XOR under $[\cdot]$-sharing before resharing (c.f., XOR-then-Resharing). Thus, we save preprocessing communication while keeping the online costs unchanged.

We present protocol Π_{PPA} for 3PC PPA in Figure 7. Without considering the optimizations, we can guarantee the correctness of protocol Π_{PPA} based on [22]. The correctness of

Protocol Π_{AND3}

Inputs: For $i \in \{0,1,2\}$, $\mathcal{P}_i$ inputs $[\![x]\!]_i = (m_x, \langle r_x \rangle_i)$, $[\![y]\!]_i = (m_y, \langle r_y \rangle_i)$, and $[\![u]\!]_i = (m_u, \langle r_u \rangle_i)$.
Outputs: $\mathcal{P}_i$ gets $[\![z]\!]_i = (m_z, \langle r_z \rangle_i)$ with $z = x \wedge y \wedge u$.

Preprocessing Phase:
1) Parties compute correlated randomness $\langle r_{xy} \rangle = \langle r_x \rangle \wedge \langle r_y \rangle$ by using $\mathcal{F}_{\mathsf{AND2}}^{\langle\cdot\rangle}$.
2) Parties invoke $\mathcal{F}_{\mathsf{ZeroShr}}$ to let $\mathcal{P}_i$ obtain $[\alpha]_i$ with $[\alpha]_0 \oplus [\alpha]_1 \oplus [\alpha]_2 = 0$.
3) Parties invoke $\mathcal{F}_{\mathsf{Rand}}$, such that $\mathcal{P}_i$ gets random share $\langle r_z \rangle_i = ([r_z]_i, [r_z]_{i+1})$.

Online Phase:
1) Parties compute $\langle xy \rangle = m_x m_y \oplus m_x \langle r_y \rangle \oplus m_y \langle r_x \rangle \oplus \langle r_{xy} \rangle$ and $\langle u \rangle = m_u - \langle r_u \rangle$ locally, such that $\mathcal{P}_i$ gets $\langle xy \rangle_i = ([xy]_i, [xy]_{i+1})$ and $\langle u \rangle_i = ([u]_i, [u]_{i+1})$.
2) $\mathcal{P}_i$ locally computes $[z]_i = ([xy]_i \wedge [u]_i) \oplus ([xy]_{i+1} \wedge [u]_i) \oplus ([xy]_i \wedge [u]_{i+1})$.
3) $\mathcal{P}_i$ computes $[m_z]_i = [z]_i \oplus [\alpha]_i \oplus [r_z]_i$ and sends $[m_z]_i$ to $\mathcal{P}_{i-1}$ and $\mathcal{P}_{i+1}$, such that each party can reconstruct $m_z = \oplus_{i=0}^{2} [m_z]_i$.
4) $\mathcal{P}_i$ sets $[\![z]\!]_i = (m_z, \langle r_z \rangle_i)$ as outputs.

Fig. 5. Protocol for secure ANDing 3 bits.

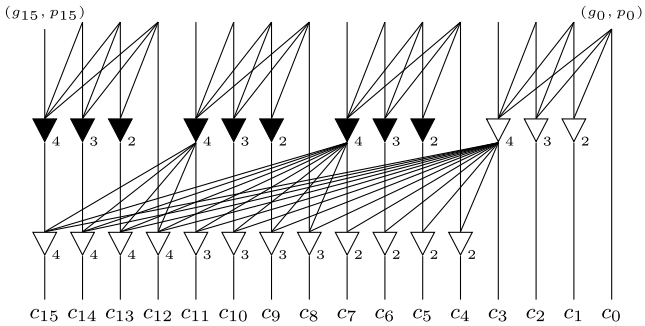


Fig. 6. 16-bit depth-optimized PPA ((g_0, p_0) to (g_{15}, p_{15})).

Protocol Π_{PPA}

Inputs: $\mathcal{P}_i$ inputs ℓ-bits Boolean sharings $[\![x]\!]_i$ and $[\![y]\!]_i$.
Outputs: $\mathcal{P}_i$ gets $[\![s]\!]_i$ with $\sum_{j=0}^{\ell-1} 2^j \cdot s_j = x + y$.

Preprocess Phase:
1) Parties jointly run the preprocessing phase of MRSS-based $\mathcal{F}_{\mathsf{AND2}}^{[\![\cdot]\!]}([\![x_j]\!], [\![y_j]\!])$ [24].
2) Parties run the preprocessing phase of secure operators $\triangledown_2$, $\blacktriangledown_2$, $\triangledown_3$, $\blacktriangledown_3$, $\triangledown_4$, and $\blacktriangledown_4$ that utilized in PPA circuit.

Online Phase:
1) Parties compute $[\![g_j]\!] = \mathcal{F}_{\mathsf{AND2}}^{[\![\cdot]\!]}([\![x_j]\!], [\![y_j]\!])$ and $[\![p_j]\!] = [\![x_j]\!] \oplus [\![y_j]\!]$ for $j \in \{0, 1, \ldots, \ell-1\}$.
2) Recursively compute the MRSS-shared intermediate signals using $([\![p]\!], [\![g]\!])$ following the depth-optimized circuit structured as Figure 6. Denote generated *carry signals* as $[\![c]\!]$.
3) Outputting the sum bits $[\![s]\!]$ with $[\![s_0]\!] = [\![p_0]\!]$ and $[\![s_j]\!] = [\![c_{j-1}]\!] \oplus [\![p_j]\!]$ when $0 < j \leq \ell - 1$.

Fig. 7. Protocol for secure parallel prefix adder.

XOR-then-Resharing can be derived from the XOR's homomorphism in 3-out-of-3 sharing and MRSS. AND-Reusing only re-uses some intermediate results of Π_{AND3} and Π_{AND4},

Authorized licensed use limited to: Nanyang Technological University Library. Downloaded on January 25,2026 at 03:17:03 UTC from IEEE Xplore. Restrictions apply.

and does not change their workflow. Mixed-AND2 adopts two existing AND2 approaches whose correctness has already been proved. So, we can guarantee the correctness of our optimized PPA.

2) Security Analysis: Functionality $\mathcal{F}_{\mathsf{PPA}}$ takes as input two MRSS Boolean sharings $[\![x]\!]$ and $[\![y]\!]$ with identifiers id_x, id_y, respectively, outputs $[\![s]\!] = [\![x+y]\!]$ corresponding to a new identifier id_s. Π_{PPA} achieves $\mathcal{F}_{\mathsf{PPA}}$ securely, its security captured in Theorem 2, and proof is given in Appendix E.

Theorem 2: [Security of Π_{PPA}] In 3-party honest-majority setting, protocol Π_{PPA} securely realizes $\mathcal{F}_{\mathsf{PPA}}$ in the ($\mathcal{F}_{\mathsf{AND2}}^{[\![\cdot]\!]}$, $\mathcal{F}_{\mathsf{AND2}}$, $\mathcal{F}_{\mathsf{AND3}}$, $\mathcal{F}_{\mathsf{AND4}}$)-hybrid model in presence of a static semi-honest adversary $\mathcal{A}$ who corrupts at most one party.

3) Round & Communication: The round and complexity of our Π_{PPA} is formulated in Lemma 2.

Lemma 2: Protocol Π_{PPA} requires 1 round with communication of $3N_3 + 3N_3' + 6N_4 + 9N_4' + 3\ell$ bits totally in the preprocessing phase, while it requires $1+\lceil\log_4(\ell)\rceil$ rounds with communication of $6(N_2+N_3+N_4) + 12(N_2'+N_3'+N_4') + 3\ell$ bits totally in the online phase. Here N_2, N_2', N_3, N_3', N_4, and N_4' denote the numbers of operators $\triangledown_2$, $\blacktriangledown_2$, $\triangledown_3$, $\blacktriangledown_3$, $\triangledown_4$, and $\blacktriangledown_4$ in the PPA circuit, $N_2 = N_3 = N_4 = \frac{\ell-1}{3}$ and $N_2' = N_3' = N_4' = \frac{\ell\log_4\ell}{4} - \frac{\ell-1}{3}$.

Proof: [Proof of Lemma 2 (Sketch)] Benefiting from AND-Reusing, we only need to generate 1 correlated triple for $\blacktriangledown_4$, 2 for $\triangledown_4$, and 3 for $\blacktriangledown_4$. The preprocessing of MRSS-based AND2 requires ℓ bits per party. All correlations can be generated in parallel. Therefore, all parties send $3N_3 + 3N_3' + 6N_4 + 9N_4' + 3\ell$ bits in 1 round during the preprocessing phase. For the online phase, each party sends 2 bits for each $\triangledown$-style operator and 4 bits for each $\blacktriangledown$-style operator. Step 1) of online phase requires each party to send ℓ bits. In total, all parties send $6(N_2+N_3+N_4) + 12(N_2'+N_3'+N_4') + 3\ell$ bits with $1+\lceil\log_4(\ell)\rceil$ rounds for online phase. For the detailed calculation of (N_2, N_3, N_4) and (N_2', N_3', N_4'), see Appendix B.■

B. Arithmetic to Boolean Conversion

Parties can use a *Boolean Adder* circuit to convert an Arithmetic share $[\![x]\!]^{\mathsf{A}}$ in $\mathbb{Z}_{2^\ell}$ to its equivalent ℓ-bits Boolean share in $\mathbb{Z}_2^\ell$. As pointed out by [9], it is more efficient to utilize the depth-optimized variant (*i.e.*, PPA) circuit to obtain a better round complexity. At a high level, given the arithmetic share $[\![x]\!]^{\mathsf{A}} = (m_x, \langle r_x\rangle^{\mathsf{A}})$, party $\mathcal{P}_0$ locally sets $u = m_x - [r_x]^{\mathsf{A}}{}_0$ followed by sharing u as ℓ-bits $[\![u]\!]$. In parallel, $\mathcal{P}_1$ generates the Boolean sharing $[\![v]\!]$ where $v = -[r_x]^{\mathsf{A}}{}_1 - [r_x]^{\mathsf{A}}{}_2$. As $u + v = m_x - [r_x]^{\mathsf{A}}{}_0 - [r_x]^{\mathsf{A}}{}_1 - [r_x]^{\mathsf{A}}{}_2 = x$, the parties can run $\Pi_{\mathsf{PPA}}([\![u]\!], [\![v]\!])$ to get $[\![x]\!] = [\![u]\!] + [\![v]\!]$. The A2B conversion is depicted as Π_{A2B} formally in Figure 8.

1) Security & Communication: Protocol Π_{A2B} only makes use of $\mathcal{F}_{\mathsf{RandPair}_{i,j}}$, $\mathcal{F}_{\mathsf{RandComm}}$, and $\mathcal{F}_{\mathsf{PPA}}$ in a black-box manner. i) We need to guarantee the security of u when $\mathcal{A}$ corrupts $\mathcal{P}_1$ or $\mathcal{P}_2$: since $[r_u]_1$ is only known to $(\mathcal{P}_0, \mathcal{P}_1)$, m_u is undistinguished from random bit-strings in $\mathbb{Z}_2^\ell$ when $\mathcal{A}$ corrupts $\mathcal{P}_2$; when $\mathcal{A}$ corrupts $\mathcal{P}_1$, we can guarantee the security of u since $[r_u]_0$ is only known to $(\mathcal{P}_0, \mathcal{P}_2)$. ii) Similarly, we can guarantee the security of v as well. The security of Π_{A2B} is easy to see in the $(\mathcal{F}_{\mathsf{RandPair}_{i,j}}, \mathcal{F}_{\mathsf{RandComm}}, \mathcal{F}_{\mathsf{PPA}})$-hybrid model.

Protocol Π_{A2B}

Inputs: $\mathcal{P}_i$ inputs Arithmetic sharing $[\![x]\!]_i^{\mathsf{A}} = (m_x, \langle r_x\rangle_i^{\mathsf{A}})$, where $\langle r_x\rangle_i^{\mathsf{A}} = ([r_x]_i^{\mathsf{A}}, [r_x]_{i+1}^{\mathsf{A}})$.
Outputs: $\mathcal{P}_i$ gets Boolean sharing $[\![x]\!]_i$ for $i \in \{0,1,2\}$.

Preprocessing Phase

1) Parties run preprocessing phase of $\mathcal{F}_{\mathsf{PPA}}([\![u]\!], [\![v]\!])$.
2) $(\mathcal{P}_0, \mathcal{P}_1)$ invoke $\mathcal{F}_{\mathsf{RandPair}_{0,1}}$ to generate $[r_u]_1 \xleftarrow{\$} \mathbb{Z}_2^\ell$, $(\mathcal{P}_0, \mathcal{P}_2)$ invoke $\mathcal{F}_{\mathsf{RandPair}_{0,2}}$ to get $[r_u]_0 \xleftarrow{\$} \mathbb{Z}_2^\ell$, and all parties run $\mathcal{F}_{\mathsf{RandComm}}$ to obtain $[r_u]_2 \xleftarrow{\$} \mathbb{Z}_2^\ell$.
3) Parties $(\mathcal{P}_i, \mathcal{P}_{i+1})$ invoke $\mathcal{F}_{\mathsf{RandPair}_{i,i+1}}$ to generate $[r_v]_{i+1} \xleftarrow{\$} \mathbb{Z}_2^\ell$ for $i \in \{0,1\}$, and all parties run $\mathcal{F}_{\mathsf{RandComm}}$ to get $[r_v]_0 \xleftarrow{\$} \mathbb{Z}_2^\ell$.

Online Phase

1) $\mathcal{P}_0$ computes $u = m_x - [r_x]_0^{\mathsf{A}}$ and broadcast $m_u = u \oplus [r_u]_0 \oplus [r_u]_1 \oplus [r_u]_2$ to all to get $[\![u]\!] = (m_u, \langle r_u\rangle)$.
2) $\mathcal{P}_1$ computes $v = -[r_x]_1^{\mathsf{A}} - [r_x]_2^{\mathsf{A}}$ and broadcast $m_v = v \oplus [r_v]_0 \oplus [r_v]_1 \oplus [r_v]_2$ to all to get $[\![v]\!] = (m_v, [r_v])$.
3) All parties run the online phase of $\mathcal{F}_{\mathsf{PPA}}([\![u]\!], [\![v]\!])$ to get $[\![x]\!] = [\![u+v]\!]$ and set $[\![x]\!]$ as outputs.

Fig. 8. Protocol for Arithmetic to Boolean conversion.

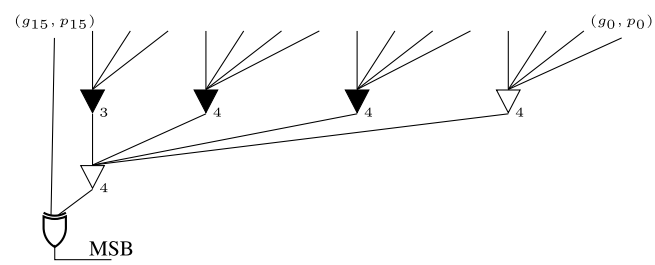


Fig. 9. Depth-optimized MSB extraction circuit for 16 bits.

Protocol Π_{A2B} results in $2+\lceil\log_4(\ell)\rceil$ online rounds and all parties need to send $6(N_2+N_3+N_4) + 12(N_2'+N_3'+N_4') + 7\ell$ bits totally. For the preprocessing phase, all parties send $3N_3 + 3N_3' + 6N_4 + 9N_4' + 3\ell$ bits in 1 round of communication.

2) Most Significant Bit Extraction: PPA can be optimized by removing unnecessary operators to extract the most significant bit from Arithmetic sharing securely (*a.k.a.*, MSB circuit) [22]. Compared to PPA circuit, MSB circuit has the same depth but requires much fewer operators. We build protocol Π_{MSB} similar to Π_{A2B} by replacing the PPA circuit of step 3) in Figure 8 with MSB circuit to extract $[\![\mathsf{MSB}(x)]\!]$. For instance, Figure 9 shows the structure of MSB circuit for 16-bit inputs, which is of depth-2, and only requires two $\triangledown_4$ and $\blacktriangledown_4$, and one $\blacktriangledown_3$, reducing approximately $5\times$ operators compared to 16-bit PPA.

C. Boolean to Arithmetic Conversion

It may also be required to convert a ℓ-bit value of Boolean sharing to an Arithmetic sharing in $\mathbb{Z}_{2^\ell}$. We can use PPA to achieve this conversion efficiently as well. In the preprocessing phase, we let $(\mathcal{P}_0, \mathcal{P}_1)$ sample random bit-string $u \xleftarrow{\$} \mathbb{Z}_2^\ell$, and $(\mathcal{P}_1, \mathcal{P}_2)$ generate a random bit-string $v \xleftarrow{\$} \mathbb{Z}_2^\ell$. Instead of directly letting $\mathcal{P}_1$ share $(-u-v)$ to get $[\![-u-v]\!]$

Authorized licensed use limited to: Nanyang Technological University Library. Downloaded on January 25,2026 at 03:17:03 UTC from IEEE Xplore. Restrictions apply.

Protocol Π_{B2A}

Inputs: $\mathcal{P}_i$ inputs Boolean sharing $[\![x]\!]_i = (m_x, \langle r_x \rangle_i)$, where $\langle r_x \rangle_i = ([r_x]_i, [r_x]_{i+1})$.
Outputs: $\mathcal{P}_i$ gets Arithmetic sharing $[\![x]\!]_i^{\mathsf{A}}$ for $i \in \{0, 1, 2\}$.

Preprocessing Phase:

1) $(\mathcal{P}_0, \mathcal{P}_1)$ run $\mathcal{F}_{\mathsf{RandPair}_{0,1}}$ to get random $u \xleftarrow{\$} \mathbb{Z}_2^\ell$, $(\mathcal{P}_1, \mathcal{P}_2)$ execute $\mathcal{F}_{\mathsf{RandPair}_{1,2}}$ generate random $v \xleftarrow{\$} \mathbb{Z}_2^\ell$, such that $[\![u \oplus v]\!] = (m = 0, \langle r \rangle = (0, u, v))$.
2) $(\mathcal{P}_0, \mathcal{P}_2)$ invoke $\mathcal{F}_{\mathsf{RandPair}_{0,2}}$ to get random $a \xleftarrow{\$} \mathbb{Z}_{2^\ell}$, $(\mathcal{P}_0, \mathcal{P}_1)$ call $\mathcal{F}_{\mathsf{RandPair}_{0,1}}$ to obtain random $b \xleftarrow{\$} \mathbb{Z}_{2^\ell}$, and $\mathcal{P}_1$ sends $c = (u \oplus v) - b$ to $\mathcal{P}_2$.
3) Parties run preprocessing phase of $\mathcal{F}_{\mathsf{PPA}}([\![x]\!], [\![u \oplus v]\!])$.

Online Phase:

1) Parties jointly evaluate $\mathcal{F}_{\mathsf{PPA}}([\![x]\!], [\![u \oplus v]\!])$ to get $[w] = [x + (u \oplus v)]$ in 3-out-of-3 sharing, and directly reveal w to $\mathcal{P}_0$ and $\mathcal{P}_2$.
2) $\mathcal{P}_0$ and $\mathcal{P}_2$ locally compute $w + a$, and reveal it to $\mathcal{P}_1$.
3) $\mathcal{P}_i$ sets $[\![x]\!]_i^{\mathsf{A}} = (w + a, \langle a + (u \oplus v) \rangle_i^{\mathsf{A}})$ as outputs.

Fig. 10. Protocol for Boolean to Arithmetic conversion.

as ABY3 [9], we observe $[\![u \oplus v]\!]$ can be expressed as $(m = 0, \langle r \rangle = (0, u, v))$. At the same time, we let $(\mathcal{P}_0, \mathcal{P}_2)$ sample random value $a \xleftarrow{\$} \mathbb{Z}_{2^\ell}$, $(\mathcal{P}_0, \mathcal{P}_1)$ sample $b \xleftarrow{\$} \mathbb{Z}_{2^\ell}$, and $\mathcal{P}_1$ compute and send $c = (u \oplus v) - b$ to $\mathcal{P}_2$, such that we get a valid $\langle a + (u \oplus v) \rangle^{\mathsf{A}} = (a, b, c)$. We save the communication costs of invoking one sharing procedure at the cost of sending c (but still save ℓ bits for preprocessing communication).

During the online phase, given $[\![x]\!]$, the parties first execute $\Pi_{\mathsf{PPA}}([\![x]\!], [\![u \oplus v]\!])$ to get $[\![w]\!] = [\![x + (u \oplus v)]\!]$ and reveal w to both $\mathcal{P}_0$ and $\mathcal{P}_2$. Then, $\mathcal{P}_0$ computes and sends $w + a$ to $\mathcal{P}_1$ ($\mathcal{P}_2$ computes $w + a$ locally). Finally, the result can be set as $[\![x]\!]^{\mathsf{A}} = (w + a, \langle a + (u \oplus v) \rangle^{\mathsf{A}})$. Besides, we observe that for Π_{PPA}, instead of generating MRSS-shared $[\![w]\!]^{\mathsf{A}}$, we modify its last step of PPA circuit to output $[\cdot]$-shared carry signals and then directly reveal w to $\mathcal{P}_0$ and $\mathcal{P}_2$ from 3-out-of-3 sharing. In this way, we can save one round of communication for generating $[\![w]\!]$. Protocol Π_{B2A} is illustrated in Figure 10.

1) Security & Communication: We analyze the security of protocol Π_{B2A} in two cases: i) When adversary $\mathcal{A}$ corrupts $\mathcal{P}_0$, it will get $w = x + (u \oplus v)$ in clear. Since v is only known to $(\mathcal{P}_1, \mathcal{P}_2)$, w is uniformly random to $\mathcal{A}$. Similarly, when $\mathcal{A}$ corrupts $\mathcal{P}_2$, we can also guarantee the uniformity of w since $\mathcal{P}_2$ does not know u. ii) In case of $\mathcal{A}$ corrupting $\mathcal{P}_1$, it will see $w + a$. As long as $(\mathcal{P}_0, \mathcal{P}_2)$ keep the secrecy of a, $w + a$ is undistinguished from a random value for $\mathcal{A}$. Other steps are either local operators or invoking existing protocols, *i.e.*, $\mathcal{F}_{\mathsf{RandPair}_{i,j}}$ and $\mathcal{F}_{\mathsf{PPA}}$, in the black-box manner. Therefore, protocol Π_{B2A} is secure in $(\mathcal{F}_{\mathsf{RandPair}_{i,j}}, \mathcal{F}_{\mathsf{PPA}})$-hybrid model.

Protocol Π_{B2A} requires $2 + \lceil \log_4 \ell \rceil$ online rounds, and all parties send $4(N_2 + N_3 + N_4) + 12(N_2' + N_3' + N_4') + 4\ell$ bits totally. For the preprocessing phase, all parties send $3N_3 + 3N_3' + 6N_4 + 9N_4' + 3\ell$ bits to generate the correlated randomness for Π_{PPA}, and $\mathcal{P}_1$ sends ℓ bits to $\mathcal{P}_2$, which can be executed in parallel (1 round communication).

TABLE III

PROFILING COMMUNICATION (KB), ROUNDS, AND TIME (SECONDS) OF A2B, B2A, MSB-EXT, AND TOTAL COSTS OF FUNCTIONS RELU AND SOFTMAX USING ABY3 (EXCEPT A2B, B2A AND MSB-EXT, TOTAL COSTS INCLUDE ARITHMETIC ADDITION, MULTIPLICATION, AND ETC. [10]).THE INPUT IS OF BATCHSIZE $|B| = 1000$

ReLU	# Call	Comm.	Round	Time
A2B	0	0	0	0
MSB-Ext	1	70.300	8	0.363
B2A	0	0	0	0
Total	-	101.563	10	0.507
Percentage (%)	-	69.2%	77.2%	70.9%

Softmax	# Call	Comm.	Round	Time
A2B	1	0.219	8	0.453
MSB-Ext	16	140.625	136	5.781
B2A	2	0.273	13	0.503
Total	-	610.297	236	11.387
Percentage (%)	-	23.1%	62.4%	59.9%

V. APPLICATION TO SECURE INFERENCE

Secure non-linear functions, *e.g.*, ReLU and Softmax, are widely used in MPC-based secure neural network inference [9], [11]. Existing works have proposed various approximation methods, *e.g.*, piece-wise polynomial approximation [8], [9], [10], to efficiently and accurately compute these functions. We denote by $g_{d,m} : \mathbb{R} \to \mathbb{R}$ a piece-wise function defined over $d + 1$ intervals, each represented by a polynomial of degree m as:

$$g_{d,m}(x) = \begin{cases} F_0(x), & \text{if } x \le I_1, \\ F_1(x), & \text{if } x \in (I_1, I_2), \\ \dots \\ F_d(x), & \text{if } x > I_d. \end{cases} \tag{4}$$

Each polynomial $F_i(x) = a_{i,0} + \sum_{j=1}^{m} a_{i,j} x^j$ is defined by at most $m + 1$ coefficients $\{a_{i,j} \in \mathbb{R}\}_j$. However, these approaches still require huge costs for the involved Boolean primitives. Table III profiles the online costs of ReLU and Softmax, which are the most widely used activation functions in neural networks, using ABY3 [10], one of the state-of-the-art MPC frameworks. We find share conversions and MSB-Ext account for over 60% of online rounds and time.

We integrate our proposed protocols, including Π_{A2B}, Π_{B2A}, and Π_{MSB}, into 3PC secure neural network inference backend of the SecretFlow-SPU to provide an optimized framework ALKAID. As shown in Figure 11, the inputs and outputs of secure neural network inference are in the RSS-shared format. We replace the corresponding prior RSS-based implementations for Boolean circuits with our 3PC protocols, and the arithmetic operations, *i.e.*, Matrix Multiplication and Truncation, which are defined on *Arithmetic circuit* for fixed-point values and computed using existing RSS-based protocols [9], [10], [32]. However, our protocols in § IV are designed with MRSS-shared inputs and outputs, so we need to bridge the RSS and MRSS worlds with the following modifications. Note in ALKAID, we focus on how to apply our proposed primitives to improve the online efficiency of these approaches instead of designing new approximation methods.

Authorized licensed use limited to: Nanyang Technological University Library. Downloaded on January 25,2026 at 03:17:03 UTC from IEEE Xplore. Restrictions apply.

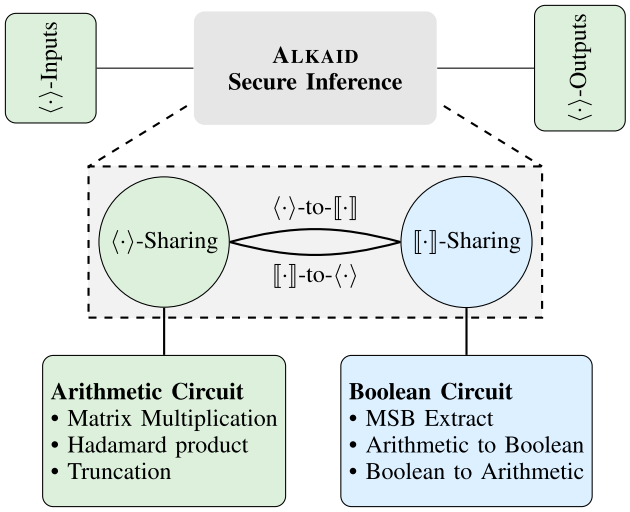


Fig. 11. Overview of the ALKAID framework, bridging RSS and MRSS for efficient secure neural network inference.

- **$\langle\cdot\rangle$-to-$[\![\cdot]\!]$ Processing.** When switching from $\langle\cdot\rangle$-sharing to $[\![\cdot]\!]$-sharing for fast Boolean circuits, we need to convert RSS-shared inputs to MRSS-shared format. For Π_{A2B}, given $\langle x\rangle^{\mathsf{A}}$, $\mathcal{P}_0$ secret-shares $[x]_0$ as $[\![u]\!]$ with $u = [x]_0$, and $\mathcal{P}_1$ secret-shares $[x]_1+[x]_2$ as $[\![v]\!]$ with $v = [x]_1+[x]_2$; For protocol Π_{MSB}, given $\langle x\rangle^{\mathsf{A}}$, the parties generate $([\![u]\!], [\![v]\!])$ from $\langle a\rangle^{\mathsf{A}}$ similar to the modifications corresponding to Π_{A2B}. ii) For Π_{B2A}, we add one *resharing* before step 1) of online phase to convert $\langle x\rangle$ to MRSS-shared $[\![x]\!]$.
- **$[\![\cdot]\!]$-to-$\langle\cdot\rangle$ Processing.** We need to convert the MRSS-shared results to the RSS world for other operations. For protocols Π_{A2B} and Π_{MSB}, we modify the lowest $\triangledown$-style operators of PPA and BitExt circuits to output RSS-shared results instead of MRSS ones; For protocol Π_{B2A}, we first let $\mathcal{P}_1$ generate $\langle u\oplus v\rangle^{\mathsf{A}}$, instead of generating (a, b, c), in step 2) of the preprocessing phase. For the online phase, when $w = x + (u \oplus v)$ is revealed to $(\mathcal{P}_0, \mathcal{P}_2)$, $(w, 0, 0)$ is of a valid RSS of $\langle x+(u\oplus v)\rangle^{\mathsf{A}}$, where $([x + (u \oplus v]_0 = w, [x + (u \oplus v]_1 = 0, [x + (u \oplus v]_2 = 0)$. In this way, the parties can locally compute $\langle x\rangle^{\mathsf{A}} = \langle x + (u \oplus v\rangle^{\mathsf{A}} - \langle u \oplus v\rangle^{\mathsf{A}}$.

The modified protocols ${\Pi_{\mathsf{A2B}}}^{\langle\cdot\rangle}$, ${\Pi_{\mathsf{MSB}}}^{\langle\cdot\rangle}$, and ${\Pi_{\mathsf{B2A}}}^{\langle\cdot\rangle}$, and their security proofs are in Appendix D. We integrate our improved primitives into SecretFlow-SPU to boost non-linear functions and finally lead to faster secure neural network inference.

Remark 2: In the secure evaluation of non-linear functions, we integrate protocols ${\Pi_{\mathsf{A2B}}}^{\langle\cdot\rangle}$, ${\Pi_{\mathsf{B2A}}}^{\langle\cdot\rangle}$, and ${\Pi_{\mathsf{MSB}}}^{\langle\cdot\rangle}$ into SecretFlow-SPU and use them to improve the online efficiency of corresponding parts. The other parts are computed using RSS. It is also possible to use our ANDN protocols and depth-optimized Π_{PPA} to accelerate the online efficiency of other parts, which we leave for future work.

VI. Implementation & Evaluation

We implement ALKAID and extensive evaluate its performance by answering the following questions.

TABLE IV

COMMUNICATION (KB) AND TIME (MILLISECONDS) OF SECURE PPA CIRCUIT, WHERE BIT-WIDTH ℓ = 64 BITS AND BATCHSIZE $|B|$ = 1 AND 1000. ALKAID - ON. IS FOR THE ONLINE PHASE, AND ALKAID - TOT. INDICATES THE TOTAL OVERHEAD

$\|B\|$	Protocol	Comm.	LAN	WAN
1	ABY3	0.203	5.015	352.078
	Trifecta	4.483	–	213.000
	ALKAID-On.	0.141	2.685	201.828
	ALKAID-Tot.	0.198	3.341	252.214
1000	ABY3	203.125	6.526	375.137
	Trifecta	4378.472	–	339.561
	ALKAID-On.	140.625	10.469	219.770
	ALKAID-Tot.	197.265	11.072	271.105

- **Q1:** What are our communication and running time advantages for depth-optimized PPA? (§ VI-B)
- **Q2:** How about our efficiency for 3PC Boolean primitives? Is it practical to support share conversions? (§ VI-C)
- **Q3:** Can ALKAID support fast secure evaluation of non-linear functions and neural network inference? (§ VI-D)

A. Experimental Setup

Testbed Environments. Experiments are run on a machine with Intel(R) Xeon(R) Silver 4314 CPU @ 2.40 GHz and 500 GB RAM. The Operating System is Ubuntu 22.04.4 LTS with Linux kernel 5.4.0-172-generic. The Wide-Area Network (WAN) environment is emulated using the Linux Traffic Control (tc) command, with configured: i) LAN: bandwidth of 1 Gbps and a round-trip latency of 2 ms. ii) WAN: bandwidth of 160 Mbps and a round-trip latency of 50 ms. Bitwidth is configured as $\ell = 64$ by default.

Baselines. We compare ALKAID to prior works including ABY3 [9], Trifecta [25] and METEOR [24], to show our improvements thoroughly: i) We implement ALKAID on top of the SecretFlow-SPU framework [10] in C++ and Python version-3.10. ii) We re-run ABY3 implemented in SecretFlow-SPU and METEOR in our tested environments. Note that ABY3 and Trifecta do not separate preprocessing and online phases (which incur almost no preprocessing costs), so their total costs are equivalent to online cost. We compare ALKAID with them in both online and total costs. As Trifecta is not publicly available, we set our tested environments to be comparable as theirs for fair comparison. iii) We count the sent and received messages per party as communication costs.

B. Benchmarks of Parallel Prefix Adder

We benchmark the online efficiency of PPA. The experiments are performed with bit-width $\ell = 64$ and batchsize $|B| \in \{1, 1000\}$, and the results are illustrated in Table IV. Note that PPA is with MRSS-shared inputs and outputs here.

As shown in Table IV, we compare our communication costs, round complexity, and running time with ABY3 and Trifecta: i) Compared to ABY3, ALKAID reduces communication costs by approximately 1.3×. In terms of running time, ALKAID generally outperforms ABY3, achieving a

Authorized licensed use limited to: Nanyang Technological University Library. Downloaded on January 25,2026 at 03:17:03 UTC from IEEE Xplore. Restrictions apply.

TABLE V
COMMUNICATION (KB) AND TIME (MILLISECONDS) OF SECURE A2B, B2A, AND MSB-EXT, WHERE BIT-WIDTH $\ell = 64$, BATCHSIZE $|B| = 1$ AND 1000

$\|B\|$	Primitive	Protocol	Comm.	LAN	WAN
1	A2B	ABY3	0.219	5.659	402.737
		ALKAID-On.	0.164	4.846	302.521
		ALKAID-Tot.	0.221	5.523	352.907
	B2A	ABY3	0.242	6.479	452.797
		ALKAID-On.	0.133	4.823	307.327
		ALKAID-Tot.	0.198	5.574	357.674
	MSB-Ext	ABY3	0.070	5.841	403.133
		METEOR-On.	0.178	10.634	310.495
		METEOR-Tot.	0.662	17.267	418.387
		ALKAID-On.	0.066	4.545	301.979
		ALKAID-Tot.	0.105	5.192	352.190
1000	A2B	ABY3	218.750	7.198	405.648
		ALKAID-On.	164.063	12.704	322.159
		ALKAID-Tot.	220.703	16.197	373.494
	B2A	ABY3	242.188	8.526	455.614
		ALKAID-On.	132.813	12.508	320.074
		ALKAID-Tot.	197.266	15.651	378.805
	MSB-Ext	ABY3	70.313	8.453	407.687
		METEOR-On.	178.176	11.980	329.943
		METEOR-Tot.	662.395	19.392	455.147
		ALKAID-On.	66.406	6.961	305.860
		ALKAID-Tot.	97.656	8.955	356.715

1.6-1.7× speedup in WAN, which aligns with its reductions in round complexity. ii) However, the running time gains in LAN are less significant due to ample bandwidth and lower latency, where reductions in communication and rounds do not lead to effective running time savings. Additionally, ALKAID even requires more running time when batchsize is 1000. This is because ALKAID requires more local AND and XOR gates in N-input AND protocol and complex circuit logic designs compared to ABY3, introducing some computational and memory access overhead. However, this impact can be mitigated through hardware optimizations. iii) Compared to Trifecta, we achieve comparable round complexity while reducing the communication costs by 28×, since Trifecta needs exponentially more communication to generate correlations during the online phase. Moreover, our approach reduces the running time by around 5%-13%.

Besides, we measure the preprocessing communication and running time in Table IV. Putting preprocessing and online costs together, we analyze the end-to-end efficiency in WAN as follows: Compared to ABY3, even we require ≈ 28% more communication, ALKAID is still 1.2-1.3× faster. Compared to Trifecta, we even reduce total communication by 17× and achieve 1.2× total time improvements in batch processing.

C. Benchmarks of Boolean Primitives

In Table V, we present the online evaluations of 3PC Boolean primitives: A2B, B2A, and MSB-Ext. We mainly compare ALKAID to ABY3: i) Similar to the evaluation analysis of PPA, ALKAID also outperforms ABY3 in online communication and running time for most cases in WAN. Concretely, we reduce the communication of A2B and B2A conversion by 1.2-1.6×, and reduce the running time by approximately 1.2-1.5×. In LAN, we achieve a comparable running time with ABY3 and introduce a little more time for batch evaluation of A2B and B2A conversion. The reason is similar to that of batch evaluation of PPA. ii) Notably, our B2A circuit requires less communication than PPA. For instance, with $\ell = 64$ and $|B| = 1000$, B2A incurs 132.813 KB in communication, whereas PPA requires 140.625 KB, as a result of the specific optimizations in protocol Π_{B2A} (see Figure 10). However, Π_{B2A} does introduce longer running time due to its deeper circuit and increased rounds. iii) For MSB-Ext, ALKAID reduces round complexity by 1.6×, allowing it to remain approximately 1.2× faster than ABY3. Compared to METEOR, our online phase is still more concretely efficient. Even we require twice online communication complexity for N-bit AND (c.f., Table I), ALKAID is 1.1-2.5× faster and reduces the concrete communication costs by 2.6× for the online phase. *The online communication reductions arise from that* METEOR *is built on* FALCON*, which requires to secret-share bit values in* $\mathbb{Z}_p$ *with prime* $p > \ell$*. As a consequence,* METEOR *requires* $\lceil \log_2 p \rceil\times$ *more concrete online communication.*

Table V also includes the end-to-end communication and running time for each primitive. Our total communication is also comparable or even better than ABY3 and METEOR with our specific optimizations. As METEOR does not open-source the preprocessing implementation, we implement their correlation generation. Moreover, when comparing our total costs to METEOR, we still achieve a communication reduction of approximately 7× and 1.2–2.1× faster for MSB-Ext. Besides, we reduce total running time by upto 1.3× than ABY3.

D. Performance of Secure Inference

In practical applications, the inputs are usually floating-point values. We truncate the floating-point values as fixed-point representations while maintaining f fractional bits and then encode the fixed-point values as integers within $\mathbb{Z}_{2^\ell}$. Following [10], we set $\ell = 64$ and $f = 18$. Note the inputs and outputs of non-linear functions and secure neural networks inference are of RSS-shared format. We incorporate our modified primitives proposed in § V into the SPU implementation of ABY3 while leveraging existing ABY3-based implementations for other operations. Also, as we only modify the protocols for Boolean circuits and follow existing works for others, *e.g.*, approximation and truncation methods, ALKAID can guarantee the same level of precision and model accuracy as SecretFlow-SPU [10], so we focus on performance evaluation in this work.

1) Online Performance: First, we compare ALKAID with prior works in terms of online performance.

- **Non-linear Functions.** We select ReLU, Sigmoid, GeLU, and Softmax for evaluations, and their specifications can be found in Appendix C. Table VI illustrates the experiment results for secure activation functions, highlighting the improvements achieved by ALKAID. i) Compared to ABY3, ALKAID demonstrates higher efficiency in online running time across almost all experiments in both LAN and WAN. This is particularly due to our reduction of circuit depth. Roughly, we reduce the circuit depth

Authorized licensed use limited to: Nanyang Technological University Library. Downloaded on January 25,2026 at 03:17:03 UTC from IEEE Xplore. Restrictions apply.

TABLE VI

COMMUNICATION (KB) AND RUNNING TIME (MILLISECONDS) OF SECURE ACTIVATION FUNCTIONS FOR $|B| = 1$ AND 1000. FOR SOFTMAX, WE SET $|B| = 10$ AND 1000

$\|B\|$	Functions	Protocol	Comm.	LAN	WAN
1	ReLU	ABY3	0.102	10.003	503.214
		METEOR-On.	0.195	9.472	352.034
		METEOR-Tot.	0.678	17.381	486.033
		ALKAID-On.	0.096	9.078	403.341
		ALKAID-Tot.	0.127	9.865	453.668
	Sigmoid	ABY3	0.552	22.478	1508.344
		ALKAID-On.	0.348	19.398	1064.298
		ALKAID-Tot.	0.418	20.397	1115.638
	GeLU	ABY3	0.806	39.098	2826.476
		ALKAID-On.	0.803	40.098	2518.875
		ALKAID-Tot.	0.897	40.846	2569.745
	Softmax ($\|B\|$=10)	ABY3	7.016	101.349	6272.879
		ALKAID-On.	6.701	87.078	5550.320
		ALKAID-Tot.	7.342	87.857	5601.810
1000	ReLU	ABY3	101.562	14.087	507.467
		METEOR-On.	195.560	11.437	424.098
		METEOR-Tot.	678.776	20.073	554.283
		ALKAID-On.	93.994	14.398	409.341
		ALKAID-Tot.	125.244	16.386	461.362
	Sigmoid	ABY3	465.250	35.431	1520.087
		ALKAID-On.	344.238	32.879	1077.687
		ALKAID-Tot.	414.551	38.897	1133.763
	GeLU	ABY3	834.375	62.031	2836.887
		ALKAID-On.	797.607	52.349	2542.891
		ALKAID-Tot.	894.357	56.350	2599.922
	Softmax	ABY3	610.297	192.319	11387.271
		ALKAID-On.	563.923	160.081	9606.879
		ALKAID-Tot.	626.438	162.862	9657.776

by approximately 1.2-1.5×. Consequently, our approach reduces online running time by ≈ 1.2× in WAN on average and is more efficient than ABY3 in terms of online communication. ii) For secure ReLU, Compared to METEOR, we reduce the online communication of the function ReLU by 2.0×. However, we require slightly more running time than METEOR. This is mainly because our SPU-based implementation generates randomness faithfully, while METEOR prototype simply uses *zero* values for protocol simulation following Falcon. So, we require more computation overhead. Even though our online running time is comparable to METEOR.

- **Secure Inference.** We employ three widely used neural networks: LeNet-5 [33], ResNet-50 [34], and GPT-2 [35], and their specifications are illustrated in Appendix C. Following prior works [9], [17], we mainly focus on the communication and running time of the 3-party inference, *a.k.a.*, we do not include the model deployment cost since this can be done once in the system setup phase. For LeNet-5 (resp., ResNet-50), we input one image from the MNIST dataset [33] (resp., CIFAR-10 [36]) for image classification; for GPT-2, we input 8 tokens and generate 1 new token. Table VII shows the online efficiency of secure neural network inference. Compared to ABY3, ALKAID requires less communication cost and is faster than ABY3 by approximately 1.1-1.2×. Compared to METEOR [24], we achieve 1.35× reduction in communication and approximately 10% time savings for the online phase.

TABLE VII

COMMUNICATION (MB) AND TIME (SECONDS) OF SECURE INFERENCE OF NEURAL NETWORKS. THE INPUT OF LENET AND RESNET-50 IS 1 IMAGE AND OUTPUT IS THE LABEL. WE MEASURE GPT-2 COSTS FOR INPUTTING 8 TOKENS AND GENERATING 1 TOKEN

Model	Protocol	Comm.	LAN	WAN
LeNet-5	ABY3	2.439	0.264	4.833
	METEOR-On.	3.137	0.230	4.445
	METEOR-Tot.	7.312	0.312	5.573
	ALKAID-On.	2.327	0.224	3.982
	ALKAID-Tot.	2.791	0.237	4.079
ResNet-50	ABY3	2347.265	50.207	377.384
	ALKAID-On.	2306.462	49.671	348.023
	ALKAID-Tot.	2649.305	55.314	374.812
GPT-2	PUMA†	459.157	21.065	428.663
	ALKAID-On.	422.192	19.784	395.057
	ALKAID-Tot.	459.926	20.772	398.120

† PUMA is developed on protocol ABY3.

2) End-to-End Performance Analysis: Tables VI and VII also summarize the total costs of secure activation functions and neural network inference. Our preprocessing involves cheap communication and running time, which account for <10% of online costs for most cases. Our total running time outperforms ABY3 and METEOR for all activation functions and NNs. Compared to METEOR, we reduce the total communication by > 2.6× and total time by 1.2–1.3×. Compared to ABY3, taking secure GPT-2 as an example, we still achieve around 7% reduction in total running time in WAN. Nonetheless, our approach does not require exponential preprocessing communication, due to our improved multi-input AND gate design. ALKAID prioritizes low online latency, making it beneficial in latency-sensitive scenarios over prior works.

VII. RELATED WORK

Secure multiparty computation (MPC) protocols can be constructed from secret sharing [37], [38]. In works [3], [9], [21], [27], the parties securely evaluate one depth of gates of the circuits using the secret sharing-based subprotocols (*e.g.*, XOR and AND) until outputs. It is also possible to design MPC protocols from homomorphic encryption (HE) [39], [40] or GC [2], [20], [41], [42], Secret sharing-based protocols enjoy high-throughput since they require much less (online) overhead and allow multiple instances to be executed in parallel [21], [22].

However, secret sharing-based approaches are still limited by their communication rounds, especially in high-latency networks [21]. Existing works have exploited the function-dependent but input-independent preprocessing/online technique [22], [43] to design multi-input AND (*a.k.a.*, multiplication in arithmetic) protocols. At a high level, this approach mainly designed a masked secret sharing scheme, generated correlations according to the function topology, and finally applied the correlations to improve online efficiency. Reference [43] used the masked secret sharing over framework SPDZ [19], [44], [45] to reduce its online communication rounds and costs. [23], [26] used a dealer to distribute correlated randomness among two computing parties. Following

Authorized licensed use limited to: Nanyang Technological University Library. Downloaded on January 25,2026 at 03:17:03 UTC from IEEE Xplore. Restrictions apply.

[26], [46], [47], and [48] designed similar MPC protocols for 3 to 5 parties to resist malicious adversaries in honest-majority. ABY2.0 [22] emulated the requirements for the dealer and built a 2PC framework that supports Arithmetic, Boolean, and Yao's GC, along with their conversion. Brüggemann et al. [49] made use of ABY2.0 to improve the efficiency of secure 2PC lookup table evaluation over [50] and [51]. Trifecta [25] further constructed a more efficient dealer-based 3PC multi-input AND protocol by putting the preprocessing costs into the online phase. METEOR [24] built another 3PC replicated secret sharing-based multi-input AND protocol on top of [11] and [22]. However, the above approaches all require a communication cost exponential to the number of inputs to generate the correlation for multi-input AND. Consequently, most of them are limited to support 4-input AND, except that Trifecta tried 8-inputs AND with much more communication. Function secret sharing (FSS) [52], [53], [54] is another MPC technique that could reduce the round complexity significantly and have been used in [55], [56], and [57], but it requires huge communication costs to distribute the FSS keys.

RSS-based 3PC solutions have been applied to many practical privacy-preserving applications, such as machine learning [9], [11], [12], [17], [32], [58], [59], heavy hitters [60], graph analysis [16], [61], *etc*. Consequently, designing an RSS-based or -compatible fast multi-input AND (even with a lightweight preprocessing/correlation generation phase) and applying it to existing frameworks is meaningful.

VIII. Conclusion & Future Work

We introduce ALKAID a framework for 3-party multi-input AND gates by mixing correlations and redundancy. Following that, we construct several round-efficient 3PC primitives, including depth-optimized PPA and fast share conversions, with specific optimizations to improve efficiency concretely.

Future Work. By integrating these techniques, ALKAID accelerates secure neural network inference. Extending our protocols to arithmetic circuits for multi-input multiplication of integers is straightforward. However, it is non-trivial to extend ALKAID to multi-input multiplication of fixed-point computation since it requires one secure truncation after multiplying two fixed-point values. On the other hand, extending ALKAID to the multi-party setting is both natural and meaningful. At a high level, our multi-input AND protocol is built upon linear secret sharing with redundancy. Given such a linear secret-sharing scheme, constructing masked secret sharing is straightforward. Following the same design philosophy as in our 3PC setting, the correlated randomness required for multi-input AND gates can be efficiently generated using the underlying linear secret sharing. The parties can then leverage these correlations, together with the redundancy, to support multi-input AND protocols that achieve improved online efficiency while maintaining practical preprocessing costs.

Acknowledgment

Any opinions, findings and conclusions or recommendations expressed in these materials are those of the author(s) and do not reflect the views of the National Research Foundation, Singapore; Cyber Security Agency of Singapore; and CyberSG Research and Development Programme Office, Singapore.

References

[1] A. C. Yao, "Protocols for secure computations," in *Proc. 23rd Annu. Symp. Found. Comput. Sci. (SFCS)*, Nov. 1982, pp. 160–164.

[2] A. C.-C. Yao, "How to generate and exchange secrets," in *Proc. 27th Annu. Symp. Found. Comput. Sci. (SFCS)*, Oct. 1986, pp. 162–167.

[3] O. Goldreich, S. Micali, and A. Wigderson, "How to play ANY mental game," in *Proc. 19th Annu. ACM Conf. Theory Comput. (STOC)*, 1987, pp. 218–229.

[4] D. Bogdanov, S. Laur, and J. Willemson, "Sharemind: A framework for fast privacy-preserving computations," in *Proc. Eur. Symp. Res. Comput. Security*, Sep. 2008, pp. 192–206.

[5] S. Arora et al., "Privacy-preserving financial anomaly detection via federated learning & multi-party computation," in *Proc. Annu. Comput. Secur. Appl. Conf. Workshops (ACSAC Workshops)*, 2024.

[6] Y. Yang et al., "PriRanGe: Privacy-preserving range-constrained intersection query over genomic data," *IEEE Trans. Cloud Comput.*, vol. 11, no. 3, pp. 2379–2391, Jul. 2023.

[7] P. Mohassel and Y. Zhang, "SecureML: A system for scalable privacy-preserving machine learning," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2017, pp. 19–38.

[8] J. Liu, M. Juuti, Y. Lu, and N. Asokan, "Oblivious neural network predictions via minionn transformations," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2017, pp. 619–631.

[9] P. Mohassel and P. Rindal, "ABY3: A mixed protocol framework for machine learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Security*, Jul. 2018, pp. 35–52.

[10] J. Ma et al., "SecretFlow-SPU: A performant and user-friendly framework for privacy-preserving machine learning," in *Proc. USENIX Annu. Tech. Conf.*, Boston, MA, USA, Jul. 2023, pp. 17–33, 2023.

[11] S. Wagh, S. Tople, F. Benhamouda, E. Kushilevitz, P. Mittal, and T. Rabin, "FALCON: Honest-majority maliciously secure framework for private deep learning," 2020, *arXiv:2004.02229*.

[12] S. Tan, B. Knott, Y. Tian, and D. J. Wu, "CryptGPU: Fast privacy-preserving machine learning on the GPU," 2021, *arXiv:2104.10949*.

[13] J. Bai et al., "Mostree: Malicious secure private decision tree evaluation with sublinear communication," in *Proc. Annu. Comput. Secur. Appl. Conf.*, Dec. 2023, pp. 799–813.

[14] D. Bhardwaj, S. Saravanan, N. Chandran, and D. Gupta, "Securely training decision trees efficiently," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Dec. 2024, pp. 4673–4687.

[15] G. Lin et al., "Ents: An efficient three-party training framework for decision trees by communication optimization," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Dec. 2024, pp. 4376–4390.

[16] T. Araki, J. Furukawa, K. Ohara, B. Pinkas, H. Rosemarin, and H. Tsuchida, "Secure graph analysis at scale," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2021, pp. 610–629.

[17] Y. Dong et al., "PUMA: Secure inference of LLaMA-7B in five minutes," 2023, *arXiv:2307.12533*.

[18] H. Wu et al., "Ditto: Quantization-aware secure inference of transformers upon MPC," in *Proc. Int. Conf. Mach. Learn.*, 2024, pp. 53346–53365.

[19] M. Keller, "MP-SPDZ: A versatile framework for multi-party computation," in *Proc. ACM SIGSAC Conf. Comput. Commun. Security*, 2020, pp. 1575–1590.

[20] Y. Huang, D. Evans, J. Katz, and L. Malka, "Faster secure two-party computation using garbled circuits," in *Proc. 20th USENIX Secur. Symp.*, Sep. 2011, pp. 1–35.

[21] T. Schneider and M. Zohner, "GMW vs. Yao? Efficient secure two-party computation with low depth circuits," in *Proc. Int. Conf. Financial Cryptogr. Data Secur.* Cham, Switzerland: Springer, 2013, pp. 275–292.

[22] A. Patra, T. Schneider, A. Suresh, and H. Yalame, "ABY2.0: Improved mixed-protocol secure two-party computation," in *Proc. 30th USENIX Secur. Symp. (USENIX Secur.)*, 2021, pp. 2165–2182.

[23] S. Ohata and K. Nuida, "Communication-efficient (client-aided) secure two-party protocols and its application," in *Proc. 24th Int. Conf. Financ. Cryptogr. Data Secur.*, 2020, pp. 369–385.

[24] Y. Dong, C. Xiaojun, W. Jing, L. Kaiyun, and W. Wang, "Meteor: Improved secure 3-party neural network inference with reducing online communication costs," in *Proc. ACM Web Conf.*, Apr. 2023, pp. 2087–2098.

[25] S. Faraji and F. Kerschbaum, "Trifecta: Faster high-throughput three-party computation over WAN using multi-fan-in logic gates," *Proc. Privacy Enhancing Technol.*, vol. 2023, no. 4, pp. 224–237, Oct. 2023.

Authorized licensed use limited to: Nanyang Technological University Library. Downloaded on January 25,2026 at 03:17:03 UTC from IEEE Xplore. Restrictions apply.

[26] H. Chaudhari, A. Choudhury, A. Patra, and A. Suresh, "ASTRA: High throughput 3PC over rings with application to secure prediction," in *Proc. ACM SIGSAC Conf. Cloud Comput. Security Workshop (CCSW)*, 2019, pp. 81–92.

[27] T. Araki, J. Furukawa, Y. Lindell, A. Nof, and K. Ohara, "High-throughput semi-honest secure three-party computation with an honest majority," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Oct. 2016, pp. 805–817.

[28] J. Furukawa, Y. Lindell, A. Nof, and O. Weinstein, "High-throughput secure three-party computation for malicious adversaries and an honest majority," in *Proc. Annu. Int. Conf. Theory Appl. Cryptograph. Techn. (EUROCRYPT)*, 2017, pp. 225–255.

[29] R. A. Finkel and J. L. Bentley, "Quad trees a data structure for retrieval on composite keys," *Acta Inf.*, vol. 4, no. 1, pp. 1–9, 1974.

[30] A. Beaumont-Smith and C.-C. Lim, "Parallel prefix adder design," in *Proc. 15th IEEE Symp. Comput. Arithmetic.*, Mar. 2001, pp. 218–225.

[31] D. Harris, "A taxonomy of parallel prefix networks," in *Proc. 37th Asilomar Conf. Signals, Syst. Comput.*, Mar. 2003, pp. 2213–2217.

[32] A. Dalskov, D. Escudero, and M. Keller, "Secure evaluation of quantized neural networks," *Proc. Privacy Enhancing Technol.*, vol. 2020, no. 4, pp. 355–375, Oct. 2020.

[33] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[34] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778.

[35] A. Radford et al., "Language models are unsupervised multitask learners," *OpenAI Blog*, vol. 1, no. 8, p. 9, 2019.

[36] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," 2014, *arXiv:1409.1556*.

[37] A. Shamir, "How to share a secret," *Commun. ACM*, vol. 22, no. 11, pp. 612–613, Nov. 1979.

[38] R. Cramer, I. Damgård, and U. Maurer, "General secure multi-party computation from any linear secret-sharing scheme," in *Proc. Int. Conf. Theory Appl. Cryptograph. Techn.* Cham, Switzerland: Springer, 2000, pp. 316–334.

[39] P. Paillier, "Public-key cryptosystems based on composite degree residuosity classes," in *Proc. Int. Conf. Theory Appl. Cryptograph. Techn.*, May 1999, pp. 223–238.

[40] C. Gentry, "Fully homomorphic encryption using ideal lattices," in *Proc. 41st Annu. ACM Symp. Theory Comput.*, May 2009, pp. 169–178.

[41] V. Kolesnikov and T. Schneider, "Improved garbled circuit: Free XOR gates and applications," in *Proc. Int. Colloq. Automata, Lang., Program.*, vol. 5126. Berlin, Germany: Springer, Jul. 2008, pp. 486–498.

[42] S. Zahur, M. Rosulek, and D. Evans, "Two halves make a whole: Reducing data transfer in garbled circuits using half gates," in *Proc. 34th Annu. Int. Conf. Theory Appl. Cryptograph. Techn. Adv. Cryptol. (EUROCRYPT)*. Cham, Switzerland: Springer, 2015, pp. 220–250.

[43] A. Ben-Efraim, M. B. Nielsen, and E. Omri, "Turbospeedz: Double your online SPDZ! Improving SPDZ using function dependent preprocessing," in *Proc. 17th Int. Conf. Appl. Cryptogr. Netw. Secur.*, Bogota, Colombia, 2019, pp. 530–549.

[44] I. Damgård, V. Pastro, N. P. Smart, and S. Zakarias, "Multiparty computation from somewhat homomorphic encryption," in *Proc. Cryptol. Conf. (CRYPTO)*, 2012, pp. 643–662.

[45] I. Damgård, M. Keller, E. Larraia, V. Pastro, P. Scholl, and N. P. Smart, "Practical covertly secure MPC for dishonest majority-or: Breaking the spdz limits," in *Proc. 18th Eur. Symp. Res. Comput. Secur.* Cham, Switzerland: Springer, 2013, pp. 1–18.

[46] A. Patra and A. Suresh, "BLAZE: Blazing fast privacy-preserving machine learning," 2020, *arXiv:2005.09042*.

[47] M. Byali, H. Chaudhari, A. Patra, and A. Suresh, "FLASH: Fast and robust framework for privacy-preserving machine learning," *Proc. Privacy Enhancing Technol.*, vol. 2020, no. 2, pp. 459–480, Apr. 2020.

[48] N. Koti, M. Pancholi, A. Patra, and A. Suresh, "SWIFT: Super-fast and robust privacy-preserving machine learning," in *Proc. 30th USENIX Secur. Symp.*, 2020, pp. 2651–2668.

[49] A. Brüggemann, R. Hundt, T. Schneider, A. Suresh, and H. Yalame, "FLUTE: Fast and secure lookup table evaluations," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2023, pp. 515–533.

[50] Y. Ishai, E. Kushilevitz, S. Meldgaard, C. Orlandi, and A. Paskin-Cherniavsky, "On the power of correlated randomness in secure computation," in *Proc. 10th Theory Cryptography Conf.*, 2013, pp. 600–620.

[51] G. Dessouky, F. Koushanfar, A. Sadeghi, T. Schneider, S. Zeitouni, and M. Zohner, "Pushing the communication barrier in secure computation using lookup tables," in *Proc. NDSS*, 2017.

[52] E. Boyle, N. Gilboa, and Y. Ishai, "Function secret sharing," in *Proc. Annu. Int. Conf. Theory Appl. Cryptograph. Techn.*, 2015, pp. 337–367.

[53] E. Boyle, N. Gilboa, and Y. Ishai, "Function secret sharing: Improvements and extensions," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Oct. 2016, pp. 1292–1303.

[54] E. Boyle, N. Gilboa, and Y. Ishai, "Secure computation with preprocessing via function secret sharing," in *Proc. Theory Cryptography Conf.* Cham, Switzerland: Springer, 2019, pp. 341–371.

[55] S. Wagh, "Pika: Secure computation using function secret sharing over rings," *Proc. Privacy Enhancing Technol.*, vol. 2022, no. 4, pp. 351–377, Oct. 2022.

[56] K. Gupta et al., "SIGMA: Secure GPT inference with function secret sharing," in *Proc. Privacy Enhancing Technol. Symp. (PETS)*, vol. 2024, Jun. 2024, pp. 61–79.

[57] N. Jawalkar, K. Gupta, A. Basu, N. Chandran, D. Gupta, and R. Sharma, "Orca: FSS-based secure training and inference with GPUs," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2024, pp. 597–616.

[58] J.-L. Watson, S. Wagh, and R. A. Popa, "Piranha: A GPU platform for secure computation," *Proc. 31st USENIX Secur. Symp.*, 2022, pp. 827–844.

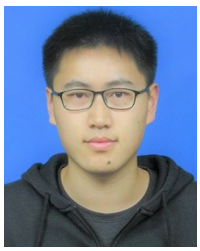
[59] Y. Li, Y. Duan, Z. Huang, C. Hong, C. Zhang, and Y. Song, "Efficient 3PC for binary circuits with application to maliciously-secure DNN inference," in *Proc. 32nd USENIX Secur. Symp.*, 2023, pp. 5377–5394.

[60] J. Kilian, A. Madeira, M. J. Strauss, and X. Zheng, "Fast private norm estimation and heavy hitters," in *Proc. 5th Theory Cryptogr. Conf. Theory Cryptogr.*, 2008, pp. 176–193.

[61] N. Koti et al., "Ruffle: Rapid 3-party shuffle protocols," *Proc. Privacy Enhancing Technol.*, vol. 2023, no. 3, pp. 24–42, 2023.

Ye Dong received the B.S. degree in computer science and technology from Shandong University in 2018 and the Ph.D. degree from the School of Cyber Security, University of Chinese Academy of Sciences. He is currently a Research Fellow with the School of Computing, National University of Singapore. His research interests include practical secure multi-party computation and privacy-preserving machine learning.

Xudong Chen received the master's degree from the University of Chinese Academy of Sciences. His research interests include privacy-preserving machine learning and private set intersection.



Xiangfu Song received the bachelor's degree in computer science and technology from Harbin Engineering University and the Ph.D. degree from Shandong University. He is currently a Research Fellow from Nanyang Technological University, Singapore. His research interests include applied cryptography and practical secure computation.

Authorized licensed use limited to: Nanyang Technological University Library. Downloaded on January 25,2026 at 03:17:03 UTC from IEEE Xplore. Restrictions apply.

Yaxi Yang received the B.S. and Ph.D. degrees in information security from the School of Cyber Security, Jinan University, Guangzhou, China, in 2017 and 2023, respectively. She is currently a Research Fellow with the School of Computer Science and Engineering, Nanyang Technological University. Her research interests include applied cryptography, practical secure multi-party computation, and private set intersection.

Wen-Jie Lu received the Ph.D. degree from the University of Tsukuba, Japan. His research interests include privacy-preserving machine learning and applied homomorphic encryption.

Tianwei Zhang (Member, IEEE) received the bachelor's degree from Peking University in 2011 and the Ph.D. degree from Princeton University in 2017. He is currently an Associate Professor and the Assistant Dean with the College of Computing and Data Science, Nanyang Technological University. His research interests include building efficient and trustworthy computer systems. He is serving on the Editorial Boards for IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY, IEEE TRANSACTIONS ON MULTIMEDIA, and IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS FOR VIDEO TECHNOLOGY.

Jianying Zhou (Fellow, IEEE) received the Ph.D. degree in information security from the Royal Holloway, University of London. He is a currently a Professor and the Center Director for iTrust, Singapore University of Technology and Design (SUTD). His research interests include applied cryptography and network security, cyber-physical system security, and mobile and wireless security. He is an ACM Distinguished Member. He received the ESORICS Outstanding Contribution Award in 2020, in recognition of his contributions to the community. He is the Co-Founder and the Steering Committee Co-Chair of ACNS. He is also the Steering Committee Chair of ACM ASIACCS and a Steering Committee Member of Asiacrypt.

Jin-Song Dong (Member, IEEE) is a Professor with the National University of Singapore. His research spans a range of fields, including formal methods with LLM agents, safety and security systems, trusted AI, probabilistic reasoning, sports analytics, and verified LLM code synthesis. He is a Fellow of the Institute of Engineers Australia.

Authorized licensed use limited to: Nanyang Technological University Library. Downloaded on January 25,2026 at 03:17:03 UTC from IEEE Xplore. Restrictions apply.